Esercizi di Programmazione

ISIS Andrea Ponti - Gallarate

Daniele Prevedello

2025 Daniele Prevedello

Table of contents

1. Esercizi di Programmazione 4
   - 1.1 Le sezioni disponibili 4
   - 1.2 Come funziona il sito 4
   - 1.3 Come usare il sito 4
2. Come usare il sito 5
   - 2.1 Navigare tra le sezioni 5
   - 2.2 Eseguire il codice con OneCompiler 5
   - 2.3 Copiare ed eseguire gli esercizi in locale 5
   - 2.4 Scaricare il repository completo 6
   - 2.5 Domande frequenti 6
3. Bash 8
   - 3.1 Bash 8
4. Linguaggio C 9
   - 4.1 Linguaggio C 9
   - 4.2 Fondamentali 13
   - 4.3 Condizioni 21
   - 4.4 Cicli 38
   - 4.5 Calcolo numerico 55
   - 4.6 Funzioni e ricorsione 59
   - 4.7 Enum 69
   - 4.8 Stringhe e vettori 71
   - 4.9 Algoritmi standard 0
5. Database 0
   - 5.1 Database e SQL 0
6. HTML-CSS-JavaScript 0
   - 6.1 HTML / CSS / JavaScript 0
7. Java 0
   - 7.1 Java 0
8. PacketTracer 0
   - 8.1 Cisco Packet Tracer e Reti 0

2025 Daniele Prevedello

9. PHP 0

9.1 PHP 0

10. Python 0

10.1 Python 0

2025 Daniele Prevedello

1. Esercizi di Programmazione

Benvenuto nella raccolta didattica di **esercizi di programmazione** pensata per gli studenti.

Qui trovi esercizi pratici, ordinati per linguaggio o argomento, con spiegazioni chiare e la possibilità di provare il codice direttamente dal browser grazie a **OneCompiler**.

1.1 Le sezioni disponibili

Sezione	Argomento	Livello indicativo
Bash	Script da riga di comando su Linux/macOS	Base → Intermedio
Linguaggio C	Linguaggio C: variabili, cicli, funzioni, puntatori	Base → Avanzato
Database	SQL: query, JOIN, DDL e DML su database relazionali	Base → Intermedio
HTML-CSS-Javascrypt	Pagine in HTML / CSS / Javascrypt	Base → Intermedio
Java	Programmazione orientata agli oggetti in Java	Intermedio → Avanzato
PacketTracer	Reti con Cisco Packet Tracer: LAN, IP, router	Base → Intermedio
PHP	PHP lato server: variabili, form, sessioni	Base → Intermedio
Python	Python: dalla sintassi base a funzioni e file	Base → Avanzato

1.2 Come funziona il sito

1. **Scegli una sezione** dal menu in alto o dalla tabella sopra.
2. **Leggi la descrizione** dell'esercizio e analizza il codice di esempio.
3. **Prova il codice** con il pulsante OneCompiler incorporato nella pagina, oppure copialo nel tuo ambiente di sviluppo locale.
4. **Consulta la soluzione** solo dopo aver provato da solo — imparare significa sbagliare e correggere!

> **Suggerimento**
>
> Se sei alle prime armi, inizia dalla sezione **Python** o **Bash**: sono i linguaggi con la sintassi più immediata.

1.3 Come usare il sito

Leggi la pagina Come usare il sito per scoprire come navigare, eseguire il codice online e scaricare i file degli esercizi.

2025 Daniele Prevedello

2. Come usare il sito

Questa pagina spiega tutto quello che ti serve per lavorare con gli esercizi.

2.1 Navigare tra le sezioni

Il sito è diviso in **sezioni**, una per ogni linguaggio o argomento. Puoi spostarti tra le sezioni in due modi:

- **Menu superiore** — clicca sul nome della sezione (Bash, C, Python, ecc.).
- **Menu laterale** — mostra tutti gli esercizi della sezione corrente.

All'interno di ogni sezione gli esercizi sono **numerati e progressivi**: conviene seguirli nell'ordine suggerito, soprattutto se sei alle prime armi.

2.2 Eseguire il codice con OneCompiler

Molte pagine includono un **bottone Prova online** con OneCompiler direttamente nella pagina: puoi scrivere e lanciare il codice senza installare nulla sul computer.

2.2.1 Come funziona

1. Il riquadro mostra un editor con il codice dell'esercizio già caricato.
2. Clicca sul pulsante ▶ **Prova online** per eseguirlo.
3. L'output appare nella parte inferiore dela pagina.
4. Puoi modificare il codice liberamente: le modifiche non vengono salvate e non influenzano gli altri studenti.

> ⚠ **Attenzione**
>
> OneCompiler richiede una **connessione a internet**. Se sei offline, copia il codice nel tuo editor locale (vedi sotto).

2.3 Copiare ed eseguire gli esercizi in locale

Se preferisci lavorare con il tuo computer, puoi copiare il codice e usarlo nel tuo ambiente di sviluppo.

2025 Daniele Prevedello

Python C Java PHP Bash

Bash

```bash
# Crea un file con il codice dell'esercizio
nano esercizio.py   # oppure usa VS Code, Thonny, ecc.

# Eseguilo
python3 esercizio.py
```

Bash

```bash
# Compila
gcc -o esercizio esercizio.c -Wall

# Esegui
./esercizio
```

Bash

```bash
# Compila
javac Esercizio.java

# Esegui
java Esercizio
```

Bash

```bash
# Esegui da riga di comando
php esercizio.php

# Oppure metti il file nella cartella del web server (es. /var/www/html/)
```

Bash

```bash
# Rendi il file eseguibile
chmod +x esercizio.sh

# Eseguilo
./esercizio.sh
```

2.4 Scaricare il repository completo

Se vuoi avere tutti gli esercizi sul computer, puoi clonare il repository con **Git**:

Bash

```bash
git clone https://github.com/nomeutente/Esercizi-programmazione.git
```

Oppure scarica lo ZIP dalla pagina del repository su GitHub: clicca su **Code → Download ZIP**.

2.5 Domande frequenti

Il codice non funziona in OneCompiler — cosa faccio?

Controlla che il linguaggio selezionato in OneCompiler corrisponda a quello dell'esercizio. Se il problema persiste, prova a eseguire il codice in locale o chiedi al tuo insegnante.

Posso salvare le mie soluzioni su OneCompiler?

Sì: crea un account gratuito su onecompiler.com e usa il pulsante **Save** per conservare i tuoi file.

2025 Daniele Prevedello

Come faccio a proporre una correzione a un esercizio?

Apri una *Issue* sul repository GitHub oppure usa il link **Modifica questa pagina** (icona matita) in cima a ogni pagina.

2025 Daniele Prevedello

3. Bash

3.1 Bash

Bash (Bourne Again SHell) è la shell predefinita di Linux e macOS. Scrivere script Bash significa automatizzare operazioni da riga di comando: copiare file, elaborare testo, eseguire programmi in sequenza e molto altro.

È un ottimo punto di partenza per capire come funziona il sistema operativo "dietro le quinte".

3.1.1 Cosa imparerai

- Eseguire comandi fondamentali del terminale (`ls`, `cd`, `cp`, `mv`, `rm`...)
- Scrivere script con variabili, cicli (`for`, `while`) e condizioni (`if`)
- Lavorare con file di testo: leggere, filtrare e trasformare l'output
- Usare pipe (`|`) e redirezione (`>`, `>>`, `<`)
- Scrivere funzioni riutilizzabili all'interno di uno script

3.1.2 Esercizi disponibili

In costruzione

Gli esercizi verranno aggiunti progressivamente. Torna a controllare questa pagina!

#	Titolo	Argomento	Difficoltà
01	*(in arrivo)*	Variabili e output	○ Base
02	*(in arrivo)*	Ciclo `for` sui file	○ Base
03	*(in arrivo)*	Condizioni `if` / `else`	◐ Intermedio
04	*(in arrivo)*	Script con parametri	◐ Intermedio
05	*(in arrivo)*	Pipe e filtri con `grep`, `awk`	● Avanzato

3.1.3 Risorse utili

- Bash Guide for Beginners — guida gratuita e completa
- explainshell.com — decostruisce qualsiasi comando
- shellcheck.net — controlla errori nel tuo script

2025 Daniele Prevedello

4. Linguaggio C

4.1 Linguaggio C

Il **linguaggio C** è uno dei linguaggi più importanti nella storia dell'informatica: nato negli anni '70, è ancora oggi usato per sistemi operativi, firmware, driver e applicazioni ad alte prestazioni.

Studiarlo significa capire davvero come funziona un programma: memoria, puntatori, tipi di dato e compilazione sono concetti fondamentali che tornano utili in qualsiasi altro linguaggio.

4.1.1 Compilare ed eseguire

Bash

```bash
gcc -o programma main.c -Wall -Wextra -lm
./programma
```

Il flag `-lm` è necessario per gli esercizi che usano `<math.h>` (seno, radice quadrata, ecc.).

4.1.2 Esercizi disponibili

In questa sezione sono raccolti una serie di esempi di programmi scritti in linguaggio C. Vanno dai più semplici ai più articolati e includono in molti casi dei commenti che ne aiutano la comprensione.

1. Fondamentali

#	Esercizio	Argomento	Difficoltà
01	Hello World	Struttura minima di un programma C, `printf`	Base
02	Media di due numeri	Variabili `float`, divisione, `scanf`	Base
03	Doppio di un numero	Variabili intere, operatori aritmetici	Base
04	Perimetro e area del quadrato	Operazioni su `float`	Base
05	Diagonale del rettangolo	Teorema di Pitagora, `sqrt`	Base
06	Circonferenza inscritta/ circoscritta	Uso di `M_PI`, `<math.h>`	Base
07	Area triangolo equilatero	Formula con `sqrt`, `<math.h>`	Base

2025 Daniele Prevedello

2. Condizioni 👤?

#	Esercizio	Argomento	Difficoltà
08	Valore assoluto	Condizione semplice, `if` / `else`	○ Base
09	Equazione della retta	Coefficiente angolare, intercetta	○ Base
10	Numero divisibile per 5	Condizione con `%`, output descrittivo	○ Base
11	Numero pari o dispari	Operatore modulo `%`, `if` / `else`	○ Base
12	Equazioni di secondo grado	Discriminante, `sqrt`, gestione casi	◑ Intermedio
13	Maggiore fra due numeri	Confronto tra variabili	○ Base
14	Maggiore fra tre numeri (v1)	`if` / `else if` annidati	○ Base
15	Maggiore fra tre numeri (v2)	Variante con variabile temporanea	○ Base
16	Maggiore fra tre numeri (v3)	Variante con funzione ausiliaria	◑ Intermedio
17	Divisibile per 2 o 3	Operatori logici `||` e `&&`	○ Base
18	Divisibile per 3 e 7	Doppia condizione con `&&`	○ Base
19	Test anno bisestile	Condizioni multiple, regola del calendario	◑ Intermedio
20	Conversione tempo in orario	Divisione intera e modulo	○ Base
21	Convertitore di temperature	Formule lineari, `switch`	◑ Intermedio
22	Stampa dei mesi	`for` + `switch`, output testuale	◑ Intermedio
23	Semplice calcolatrice	`switch`, operatori sui 4 tipi	◑ Intermedio

2025 Daniele Prevedello

3. Cicli 🔁

#	Esercizio	Argomento	Difficoltà
24	Somma dei primi 12 numeri naturali	Ciclo `for`, accumulatore	○ Base
25	Somma numeri da 10 a 20	`for` con limite variabile	○ Base
26	Vari conteggi con il for	Più contatori nello stesso ciclo	◑ Intermedio
27	Indovina il numero	`rand()`, ciclo di gioco, hint alto/basso	◑ Intermedio
28	Numero di cifre di un intero	Divisione intera ripetuta, `while`	◑ Intermedio
29	Massimo/media non definiti a priori	Ciclo con sentinella, `while`	◑ Intermedio
30	Generazione numeri casuali	`rand()`, `srand()`, `<stdlib.h>`	○ Base
31	Operazioni su numeri casuali	Min, max, conteggi su array	◑ Intermedio
32	Tavola pitagorica	Cicli `for` annidati, formattazione output	◑ Intermedio
33	Stampa rettangolo base	Cicli annidati, `*` e spazi	◑ Intermedio
34	Stampa rettangolo personalizzato	Dimensioni da input, bordi e riempimento	◑ Intermedio
35	Numeri casuali e rettangoli	Array + output grafico	◑ Intermedio
36	Disegno di figure geometriche	Triangoli, rombi, pattern con `*`	● Avanzato
37	Stampa numero binario	Maschera di bit, operatori bitwise	● Avanzato

4. Calcolo numerico 🧮

#	Esercizio	Argomento	Difficoltà
38	Stampa divisori di un numero	Ciclo con `%`, raccolta risultati	◑ Intermedio
39	Verifica numero primo	Algoritmo di primalità, `break`	◑ Intermedio
40	Calcolo del fattoriale	Funzione con ciclo `for`	○ Base
41	Sequenza di Fibonacci	Due variabili di appoggio, ciclo `for`	◑ Intermedio

5. Funzioni e ricorsione 𝑓ₓ

#	Esercizio	Argomento	Difficoltà
42	Calcolo radice quadrata	Metodo di Newton-Raphson	● Avanzato
43	Calcolo MCD	Algoritmo di Euclide iterativo	◑ Intermedio
44	Calcolo mcm (base)	Ciclo con multipli	◑ Intermedio
45	Calcolo mcm con Euclide	MCD → MCM, efficienza	● Avanzato
46	Calcolo numero di Nepero	Serie di Taylor, convergenza	● Avanzato
47	Calcolo del fattoriale (ricorsivo)	Caso base e caso ricorsivo	◑ Intermedio
48	Sequenza di Fibonacci (ricorsivo)	Doppia chiamata ricorsiva	● Avanzato

2025 Daniele Prevedello

6. Enum

#	Esercizio	Argomento	Difficoltà
49	Analisi sequenza di numeri	Lettura da input, statistiche	Intermedio

7. Stringhe e vettori

#	Esercizio	Argomento	Difficoltà
50	Conversione in maiuscolo	Array di `char`, `toupper`, `<ctype.h>`	Intermedio
51	Parola palindroma	Indici inversi, confronto caratteri	Intermedio
52	Cifrario di Cesare	Shift su caratteri, crittografia base	Intermedio
53	Statistiche su una frase	Conteggio vocali, consonanti, spazi	Intermedio
54	Conversione numero in binario	Divisioni successive, array di bit	Intermedio
55	Calcoli su vettori random	Array, somma, media su dati casuali	Intermedio
56	Conversione in binario (ricorsivo)	Ricorsione su divisione intera	Avanzato
57	Seno di un numero	Serie di Maclaurin, precisione	Avanzato

8. Algoritmi standard {...}

#	Esercizio	Argomento	Difficoltà
58	Ricerca sequenziale	Scansione lineare, flag di trovato	Intermedio
59	Bubblesort e ricerca binaria	Algoritmo $O(n^2)$, ricerca su array ordinato	Avanzato
60	Quicksort e ricerca binaria	Algoritmo O(n log n), pivot, ricorsione	Avanzato

2025 Daniele Prevedello

4.2 Fondamentali

4.2.1 Hello World

Il primo programma che si scrive in quasi tutti i linguaggi di programmazione: visualizzare un messaggio di saluto sulla console.

Obiettivo

Stampare la stringa `Hello, World!` sullo schermo.

Descrizione

"Hello, World!" è per tradizione il programma introduttivo di ogni linguaggio di programmazione. La sua semplicità lo rende ideale per verificare che l'ambiente di sviluppo funzioni correttamente e per prendere confidenza con la struttura di base di un programma in C.

Un programma in C è composto almeno da: - la direttiva `#include <stdio.h>` che include la libreria standard di input/output, - la funzione `main()` che è il punto di ingresso del programma, - la funzione `printf()` che stampa testo sulla console.

Suggerimenti

- Ogni istruzione in C termina con il punto e virgola `;`.
- La sequenza `\n` all'interno di una stringa rappresenta il carattere di a capo (newline).
- La funzione `printf()` è dichiarata nell'intestazione `<stdio.h>`: senza l'inclusione di questa libreria il programma non compila.
- Il valore restituito da `main()` con `return 0;` indica al sistema operativo che il programma è terminato senza errori.

Soluzione

C

```c
/*******************************************************************************
Classico programma introduttivo che permette di scrivere un messaggio di
saluto sulla console dell'utente.
*******************************************************************************/

#include <stdio.h>

int main(){
    printf("Hello World!");

    return 0;
}
```


2025 Daniele Prevedello

4.2.2 Media di due numeri

Dati due numeri inseriti da tastiera, il programma calcola e visualizza la loro media aritmetica.

Obiettivo

Acquisire due valori numerici dall'utente e stamparne la media aritmetica.

Descrizione

La media aritmetica di due numeri \(a\) e \(b\) è definita come:

\[m = \frac{a + b}{2}\]

Il programma deve leggere i due valori tramite `scanf()`, calcolare la media e stamparla con `printf()`.

Suggerimenti

- Usa il tipo `double` per gestire correttamente i numeri decimali: la media di 3 e 4, ad esempio, è 3.5.
- Con `scanf()`, il formato corretto per leggere un `double` è `%lf`; per stamparlo con `printf()` si usa `%f` oppure `%.2f` per limitare a due cifre decimali.
- Fai attenzione alla divisione intera: se `a` e `b` fossero dichiarati `int`, l'espressione `(a + b) / 2` troncherebbe il risultato. Dividendo per `2.0` si forza la divisione in virgola mobile.
- Prova il programma con coppie di valori di cui conosci già il risultato, ad esempio 10 e 20 (media attesa: 15.00).

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    // Dichiarazione di due variabili di tipo float (numero con la vigola)
    float numero1, numero2;

    // Lettura del primo numero
    printf("Inserire il primo numero: ");
    scanf("%f", &numero1);

    // Lettura del secondo numero
    printf("Inserire il secondo numero: ");
    scanf("%f", &numero2);

    // Calcolo la media
    float media = (numero1 + numero2) / 2;

    // Visualizzo il risultato (NOTA: con una sola cifra decimale .1f)
    printf("La media vale %.1f", media);

    return 0;
}
```


2025 Daniele Prevedello

4.2.3 Doppio di un numero

Dato un numero inserito dall'utente, il programma calcola e visualizza il suo doppio.

Obiettivo

Leggere un valore numerico da tastiera e stampare il risultato della moltiplicazione per 2.

Descrizione

Il doppio di un numero \(n\) è semplicemente:

\[d = 2 \cdot n\]

L'esercizio introduce la lettura di un valore con `scanf()`, l'esecuzione di un'operazione aritmetica elementare e la stampa del risultato con `printf()`.

Suggerimenti

- Usa il tipo `double` per poter gestire anche numeri con la virgola (ad esempio, il doppio di 3.5 è 7.0).
- Puoi calcolare il doppio direttamente all'interno di `printf()`, senza usare una variabile aggiuntiva: `printf("%.2f", 2 * n);`.
- Ricorda il formato `%lf` per leggere un `double` con `scanf()`.
- Verifica il programma con valori semplici: inserendo 5, il risultato atteso è 10.00.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    // Dichiarazione di una variabile di tipo int (numero intero)
    int numero;

    // Lettura del numero
    printf("Inserire un numero: ");
    scanf("%d", &numero);

    // Calcolo e stampo il doppio direttamente nella printf
    printf("Il doppio vale %d\n", numero*2);

    return 0;
}
```


2025 Daniele Prevedello

4.2.4 Perimetro e area del quadrato

Dato il lato di un quadrato, il programma calcola e stampa il suo perimetro e la sua area.

Obiettivo

Leggere la misura del lato di un quadrato e calcolarne perimetro e area.

Descrizione

Il quadrato è un poligono regolare con quattro lati uguali e quattro angoli retti. Date le sue proprietà, le formule per perimetro e area sono:

\[P = 4 \cdot l\]
\[A = l^2\]

dove \(l\) è la misura del lato.

Suggerimenti

- Usa `double` per il lato: un quadrato può avere lato non intero (ad esempio 2.5 cm).
- Per elevare al quadrato in C non esiste un operatore dedicato: usa semplicemente `lato * lato`. La funzione `pow(lato, 2)` dalla libreria `<math.h>` è un'alternativa, ma per il quadrato la moltiplicazione diretta è più efficiente.
- Struttura l'output in modo chiaro, stampando prima il perimetro e poi l'area su righe separate.
- Prova con lato = 5: perimetro atteso 20.00, area attesa 25.00.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int lato;

    printf("Inserire il lato: ");
    scanf("%d", &lato);

    // Calcolo area e perimetro
    int perimetro = lato*4;
    int area = lato*lato;

    printf("Il perimetro vale %d\n", perimetro);
    printf("L'area vale %d\n", area);

    return 0;
}
```


2025 Daniele Prevedello

4.2.5 Diagonale del rettangolo

Dati l'area e l'altezza di un rettangolo, il programma determina la lunghezza della diagonale.

Obiettivo

Ricavare la base del rettangolo dall'area e dall'altezza, quindi calcolare la diagonale con il teorema di Pitagora.

Descrizione

La diagonale di un rettangolo è il segmento che collega due vertici opposti. Per calcolarla si applica il teorema di Pitagora al triangolo rettangolo formato dalla base, dall'altezza e dalla diagonale stessa:

$$d = \sqrt{b^2 + h^2}$$

Poiché l'area è nota ($A = b \cdot h$), la base si ricava come:

$$b = \frac{A}{h}$$

Suggerimenti

- Includi la libreria `<math.h>` per usare la funzione `sqrt()`, che calcola la radice quadrata.
- Non dimenticare di ricavare prima la base dalla formula $b = A / h$ prima di calcolare la diagonale.
- Presta attenzione al caso in cui l'altezza sia zero: causerebbe una divisione per zero. Per ora puoi supporre che l'utente inserisca valori validi.
- Prova con area = 12 e altezza = 3: la base è 4, la diagonale è $\sqrt{16 + 9} = 5.0000$.

Soluzione

C

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    int area, altezza;

    printf("Inserire l'area del rettangolo: ");
    scanf("%d", &area);
    printf("Inserire l'altezza: ");
    scanf("%d", &altezza);

    // Inizio con il ricavare la base (per evitare la divisione intera,
    // eseguo il type-cast sulla variabile area, convertendola in float)
    float base = (float)area / altezza;

    // Calcolo della diagonale (NOTA: viene usata la funzione sqrt per il
    // calcolo della radice qudrata)
    float diagonale = sqrt(base*base + altezza*altezza);

    printf("La diagonale vale %.3f\n", diagonale);

    return 0;
}
```


2025 Daniele Prevedello

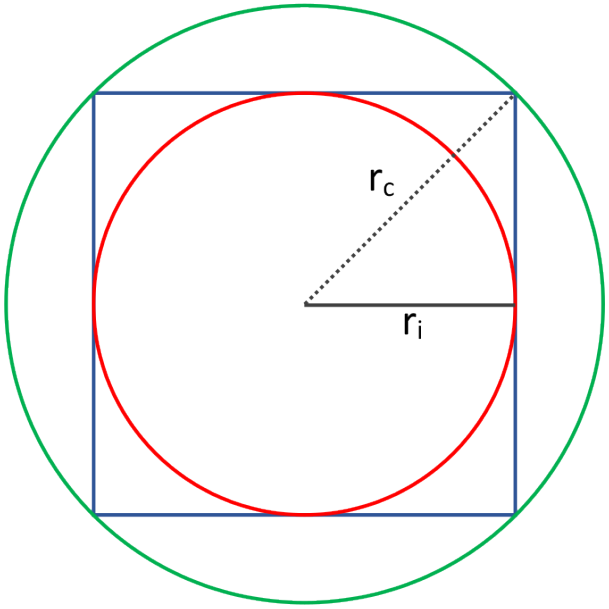
4.2.6 Circonferenza inscritta e circoscritta

Dato il lato di un quadrato, il programma calcola il raggio della circonferenza inscritta e quello della circonferenza circoscritta.

Obiettivo

Determinare i raggi delle due circonferenze associate a un quadrato, usando le proprietà geometriche del poligono regolare.

Descrizione



La **circonferenza inscritta** in un quadrato è quella tangente ai quattro lati: il suo centro coincide con il centro del quadrato e il suo raggio è pari alla metà del lato:

$$r_i = \frac{l}{2}$$

La **circonferenza circoscritta** è quella che passa per i quattro vertici: il suo raggio è pari alla metà della diagonale del quadrato. Poiché la diagonale vale $l\sqrt{2}$, si ha:

$$r_c = \frac{l\sqrt{2}}{2}$$

Suggerimenti

- Includi `<math.h>` per usare la costante `sqrt(2.0)`.
- Le due formule sono semplici: derivano direttamente da metà lato e metà diagonale del quadrato.
- Verifica il programma con lato = 4: raggio inscritto atteso 2.0000, raggio circoscritto atteso ≈ 2.8284.
- Nota che il raggio circoscritto è sempre maggiore di quello inscritto: il rapporto tra i due è costante e vale $\sqrt{2}$.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    int lato;

    printf("Inserire il lato del quadrato: ");
    scanf("%d", &lato);

    // Il raggio della circonferenza inscritta è semplicemente la metà
    // del lato del quadrato
    float raggioIns = lato / 2.0;

    // Il raggio della circonferenza circoscritta è la metà della diagonale
    // del quadrato. Con qualche passaggio matematico si ottiene:
    // float raggioCirc = sqrt(lato*lato + lato*lato) / 2;
    // float raggioCirc = sqrt(2*lato*lato) / 2;
    // float raggioCirc = lato * sqrt(2) / 2;
    float raggioCirc = lato / sqrt(2);

    printf("Il raggio inscritto vale %.1f\n", raggioIns);
    printf("Il raggio circoscritto vale %.3f\n", raggioCirc);

    return 0;
}
```


2025 Daniele Prevedello

4.2.7 Area del triangolo equilatero

Dato il perimetro di un triangolo equilatero, il programma ne calcola l'area.

Obiettivo

Ricavare il lato dal perimetro e poi calcolare l'area del triangolo equilatero con la formula apposita.

Descrizione

Un triangolo equilatero ha tutti e tre i lati uguali. Conoscendo il perimetro P, il lato vale:

$$l = \frac{P}{3}$$

L'area si calcola con la formula:

$$A = \frac{\sqrt{3}}{4} \cdot l^2$$

Questa formula si ricava applicando il teorema di Pitagora per determinare l'altezza del triangolo, che vale $h = \frac{l\sqrt{3}}{2}$, e poi usando la formula generale dell'area $A = \frac{b \cdot h}{2}$.

Suggerimenti

- Includi `<math.h>` per usare `sqrt(3.0)`.
- Il calcolo è sequenziale: prima si ricava il lato, poi si applica la formula dell'area.
- Prova con perimetro = 12: il lato è 4, l'area attesa è $\frac{\sqrt{3}}{4} \cdot 16 \approx 6.9282$.
- Il risultato ha molte cifre decimali: usa `%.4f` per stampare con quattro decimali, sufficiente per questo tipo di esercizio.

Soluzione

C

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    int perimetro;

    printf("Inserire il perimetro del triangolo equilatero: ");
    scanf("%d", &perimetro);

    float lato = perimetro / 3.0;

    // Con qualche passaggio matematico si ottiene l'altezza:
    // float altezza = sqrt(lato*lato - (lato/2.0)*(lato/2.0));
    // float altezza = sqrt(lato*lato - lato*lato/4.0);
    // float altezza = sqrt((3/4.0) * lato*lato);
    // float altezza = lato * sqrt(3/4.0);
    float altezza = lato * sqrt(3) / 2;

    float area = lato * altezza / 2;

    printf("L'area vale %.3f\n", area);

    return 0;
}
```


2025 Daniele Prevedello

4.3 Condizioni

4.3.1 Valore assoluto

Dato un numero inserito da tastiera, il programma ne stampa il valore assoluto.

Obiettivo

Leggere un numero reale e visualizzarne il valore assoluto, usando una struttura condizionale `if`.

Descrizione

Il valore assoluto di un numero \(n\) è la sua distanza dallo zero sulla retta dei numeri: è sempre non negativo. Formalmente:

\[|n| = \begin{cases} n & \text{se } n \geq 0 \\ -n & \text{se } n < 0 \end{cases}\]

Questo è uno dei primi esempi di utilizzo della struttura condizionale `if`: il programma deve prendere una decisione in base al segno del numero inserito.

Suggerimenti

- La libreria `<math.h>` fornisce la funzione `fabs()` che calcola direttamente il valore assoluto di un `double`. In questo esercizio è però utile implementarlo manualmente con un `if` per esercitarsi con la struttura condizionale.
- Se il numero è negativo, basta cambiarne il segno moltiplicandolo per `-1` (o scrivendolo come `-n`).
- Se il numero è già positivo o zero, non serve fare nulla.
- Prova con -7.5: il risultato atteso è 7.50. Prova anche con 0 e con un numero positivo.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    printf("Inserire un numero: ");
    scanf("%d", &numero);

    if(numero<0)
        numero = -numero;

    printf("Il valore assoluto è: %d\n", numero);

    return 0;
}
```


2025 Daniele Prevedello

4.3.2 Equazione della retta

Data l'equazione di una retta nella forma $y = mx + q$ e le coordinate di un punto, il programma stabilisce se il punto appartiene alla retta.

Obiettivo

Leggere i parametri m e q della retta e le coordinate (x, y) di un punto, quindi verificare se il punto soddisfa l'equazione.

Descrizione

Una retta nel piano cartesiano è descritta dall'equazione esplicita:

$$y = mx + q$$

dove m è il **coefficiente angolare** (pendenza della retta) e q è il **termine noto** (intercetta con l'asse y).

Un punto $P = (x_0, y_0)$ appartiene alla retta se e solo se le sue coordinate soddisfano l'equazione, cioè:

$$y_0 = m \cdot x_0 + q$$

Il programma deve leggere m, q, x_0 e y_0, calcolare il valore $m \cdot x_0 + q$ e confrontarlo con y_0.

Suggerimenti

- Con i numeri in virgola mobile, il confronto diretto `y == m * x + q` può essere impreciso a causa degli errori di arrotondamento. È preferibile verificare che la differenza in valore assoluto sia inferiore a una soglia piccola, ad esempio `fabs(y - (m * x + q)) < 1e-9`. Questo richiede `#include <math.h>`.
- Struttura l'output in modo chiaro: stampa le coordinate del punto e comunica esplicitamente se appartiene o no alla retta.
- Prova con $m = 2$, $q = 1$, punto $(3, 7)$: il valore della retta in $x = 3$ è $2 \cdot 3 + 1 = 7$, quindi il punto appartiene alla retta.
- Prova anche con un punto che non appartiene, ad esempio $(3, 8)$ con gli stessi parametri.

Soluzione

C

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    float m, q, xp, yp;

    printf("Inserire il coefficiente angolare m della retta: ");
    scanf("%f", &m);
    printf("Inserire la quota all'origine q della retta: ");
    scanf("%f", &q);

    printf("Inserire la coordinata x del punto: ");
    scanf("%f", &xp);
    printf("Inserire la coordinata y del punto: ");
    scanf("%f", &yp);

    float yCalcolato = m*xp+q;

    // Viene confrontato il valore di y calcolato con quello del punto
    // per determinare se esso giace o meno sulla retta
    if(yp == yCalcolato)
        printf("Il punto appartiene alla retta\n");
    else
        printf("Il punto NON appartiene alla retta\n");

    return 0;
}
```


2025 Daniele Prevedello

4.3.3 Numero divisibile per 5

Dato un numero intero, il programma comunica all'utente se è divisibile per 5, ma solo in caso affermativo.

Obiettivo

Leggere un numero intero e stampare un messaggio esclusivamente se il numero è divisibile per 5.

Descrizione

Un numero intero \(n\) è divisibile per 5 se il resto della divisione intera per 5 è zero:

\[n \mod 5 = 0\]

In C, l'operatore modulo `%` calcola il resto della divisione intera. Se `n % 5 == 0`, il numero è divisibile per 5 e si stampa il messaggio; altrimenti il programma termina senza produrre output.

Questo esercizio introduce l'uso di `if` senza `else`: la struttura condizionale esegue un'azione solo al verificarsi di una condizione, ignorando il caso contrario.

Suggerimenti

- L'operatore `%` funziona solo con operandi di tipo intero (`int`, `long`): non usarlo con `float` o `double`.
- Il numero 0 è divisibile per qualsiasi numero diverso da zero: verificalo inserendo 0 nel programma.
- Prova con 15 (divisibile), con 7 (non divisibile) e con -10 (divisibile: il modulo in C può restituire un resto negativo, ma il confronto con 0 funziona correttamente).
- Per esercizio, prova a estendere il programma aggiungendo un `else` che stampa un messaggio anche nel caso di non divisibilità.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    printf("Inserire un numero: ");
    scanf("%d", &numero);

    // Testo se il numero inserito è divisibile per 5. Per farlo calcolo il resto
    // della divisione intera per 5 tramite l'operatore % e, se pari a 0, allora
    // il numero è divisibile per 5.
    if(numero%5 == 0)
        printf("Il numero è divisibile per 5!\n");

    return 0;
}
```


2025 Daniele Prevedello

4.3.4 Numero pari o dispari

Dato un numero intero inserito dall'utente, il programma determina se è pari o dispari e comunica il risultato.

Obiettivo

Leggere un numero intero da tastiera e stampare se esso è pari o dispari.

Descrizione

Un numero intero è **pari** se il resto della sua divisione per 2 è uguale a zero, **dispari** altrimenti. In C, il resto della divisione intera si calcola con l'operatore `%` (modulo): se `n % 2 == 0` allora il numero è pari.

Suggerimenti

- Usa `scanf` per acquisire un intero e `printf` per stampare il risultato.
- L'operatore `%` restituisce il resto della divisione intera: verifica che `n % 2` sia uguale a `0`.
- Utilizza un costrutto `if-else` per gestire i due casi (pari / dispari).
- Considera anche il caso del numero zero: per convenzione è considerato pari.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
  int numero;

    printf("Inserire un numero: ");
    scanf("%d", &numero);

    // Testo se il numero inserito è divisibile per 2. Per farlo calcolo il resto
    // della divisione intera per 2 tramite l'operatore %. Se il resto è 0 allora
    // il numero è pari, altrimenti dispari.
    if(numero%2 == 0)
        printf("Il numero è pari!\n");
    else
        printf("Il numero è dispari!\n");

    return 0;
}
```


2025 Daniele Prevedello

4.3.5 Equazioni di secondo grado

Il programma richiede all'utente i coefficienti di un'equazione di secondo grado e ne determina il numero di soluzioni e i relativi valori.

Obiettivo

Acquisire i coefficienti a, b, c di un'equazione di secondo grado $ax^2 + bx + c = 0$, calcolare il discriminante e stampare le soluzioni, se esistono.

Descrizione

Data un'equazione di secondo grado nella forma $ax^2 + bx + c = 0$, il numero di soluzioni dipende dal valore del **discriminante**:

$$\Delta = b^2 - 4ac$$

- Se $\Delta > 0$: due soluzioni distinte $x_1 = \dfrac{-b + \sqrt{\Delta}}{2a}$ e $x_2 = \dfrac{-b - \sqrt{\Delta}}{2a}$
- Se $\Delta = 0$: una soluzione (due coincidenti) $x = \dfrac{-b}{2a}$
- Se $\Delta < 0$: nessuna soluzione reale

Ad esempio: $x^2 - 3x + 2 = 0$ ha due soluzioni, $x^2 - 2x + 1 = 0$ ha una soluzione (due coincidenti), $x^2 + x + 1 = 0$ non ha soluzioni.

Suggerimenti

- Includi `<math.h>` e usa la funzione `sqrt()` per calcolare la radice quadrata del discriminante.
- Gestisci il caso particolare $a = 0$: l'equazione non è di secondo grado (potresti segnalare un errore o trattarla come equazione di primo grado).
- Calcola prima il discriminante e usa `if-else` annidati per i tre casi ($\Delta > 0$, $\Delta = 0$, $\Delta < 0$).
- Quando compili, ricorda di collegare la libreria matematica con il flag `-lm`.

Soluzione

C

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    float a, b, c;

    printf("Inserire il coefficiente a dell'equazione: ");
    scanf("%f", &a);
    printf("Inserire il coefficiente b dell'equazione: ");
    scanf("%f", &b);
    printf("Inserire il coefficiente c dell'equazione: ");
    scanf("%f", &c);

    float delta = b*b - 4*a*c;

    if( delta < 0 )
        printf("Non esistono soluzioni all'equazione!\n");
    else if( delta == 0 )
        printf("Esiste una sola soluzione pari a: %.2f\n", -b/(2*a));
    else
        printf("Esistono due soluzioni: %.2f e %.2f\n", (-b+sqrt(delta))/(2*a), (-b-sqrt(delta))/(2*a));

    return 0;
}
```


2025 Daniele Prevedello

4.3.6 Maggiore fra due numeri

Dati due numeri inseriti dall'utente, il programma stampa il maggiore dei due.

Obiettivo

Leggere due numeri da tastiera e visualizzare quello con il valore più alto.

Descrizione

Il confronto tra due valori è uno dei costrutti fondamentali della programmazione. Si acquisiscono i due numeri con `scanf`, poi si utilizza un `if-else` per stabilire quale dei due è maggiore e stamparlo. È opportuno gestire anche il caso in cui i due valori siano uguali.

Suggerimenti

- Usa variabili di tipo `float` o `double` se vuoi permettere valori decimali, oppure `int` se lavori solo con interi.
- Considera il caso in cui i due numeri siano uguali e comunicalo all'utente.
- Puoi risolvere il problema anche con una variabile d'appoggio: inizializza `maggiore = a`, poi controlla se `b > maggiore` e, in quel caso, aggiorna `maggiore = b`.
- Questa strategia con la variabile d'appoggio risulta particolarmente utile quando in seguito si dovrà confrontare un numero maggiore di valori.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero1, numero2;

    printf("Inserire il primo numero: ");
    scanf("%d", &numero1);
    printf("Inserire il secondo numero: ");
    scanf("%d", &numero2);

    if(numero1 > numero2)
        printf("Il maggiore è %d!\n", numero1);
    else
        printf("Il maggiore è %d!\n", numero2);

    return 0;
}
```


2025 Daniele Prevedello

4.3.7 Maggiore fra tre numeri (versione 1)

Dati tre numeri inseriti dall'utente, il programma stampa il maggiore. In questa versione si utilizza il costrutto `if-else` annidato.

Obiettivo

Leggere tre numeri da tastiera e visualizzare il valore più alto tra i tre, usando `if-else` annidati.

Descrizione

Quando si devono confrontare tre valori, una delle strategie possibili è l'**annidamento di costrutti `if-else`**: si confronta prima il primo numero con gli altri due, e se non è il maggiore si confrontano tra loro i rimanenti. La struttura ad albero che ne risulta copre tutti i casi possibili.

In questa prima versione il codice usa esclusivamente `if-else` annidati, senza variabili d'appoggio.

Suggerimenti

- Inizia confrontando `a` con `b` e con `c`: se `a >= b && a >= c`, allora `a` è il maggiore.
- Nel ramo `else` del primo `if`, sai già che `a` non è il massimo: confronta quindi soltanto `b` e `c`.
- Fai attenzione al caso in cui due o tutti e tre i valori siano uguali: il programma deve comunque produrre un risultato corretto.
- Confronta questa soluzione con le versioni 2 e 3 per apprezzare i diversi approcci al medesimo problema.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero1, numero2, numero3;

    printf("Inserire il primo numero: ");
    scanf("%d", &numero1);
    printf("Inserire il secondo numero: ");
    scanf("%d", &numero2);
    printf("Inserire il terzo numero: ");
    scanf("%d", &numero3);

    // Verifico quale sia il maggiore fra i primi due numeri
    // quindi confronto il maggiore con il terzo
    if(numero1 > numero2)
        if(numero1 > numero3)
            printf("Il maggiore è %d!\n", numero1);
        else
            printf("Il maggiore è %d!\n", numero3);
    else
        if(numero2 > numero3)
            printf("Il maggiore è %d!\n", numero2);
        else
            printf("Il maggiore è %d!\n", numero3);

    return 0;
}
```


2025 Daniele Prevedello

4.3.8 Maggiore fra tre numeri (versione 2)

Dati tre numeri inseriti dall'utente, il programma stampa il maggiore. In questa versione si utilizza una variabile d'appoggio per memorizzare il risultato parziale.

Obiettivo

Leggere tre numeri da tastiera e visualizzare il valore più alto tra i tre, usando una variabile d'appoggio chiamata `maggiore`.

Descrizione

In questa seconda versione si introduce la variabile d'appoggio `maggiore`: si inizializza con il valore del primo numero, poi si confronta con ciascuno degli altri. Se un numero successivo risulta più grande, `maggiore` viene aggiornata. Al termine del confronto, `maggiore` contiene il valore massimo tra i tre.

Questo approccio è più scalabile rispetto all'`if-else` annidato: se in futuro si volesse trovare il massimo tra dieci numeri, basterebbe aggiungere ulteriori confronti senza modificare la struttura del codice.

Suggerimenti

- Inizializza `maggiore = a` prima di qualsiasi confronto.
- Confronta `maggiore` con `b`: se `b > maggiore`, aggiorna `maggiore = b`.
- Ripeti il confronto con `c`: se `c > maggiore`, aggiorna `maggiore = c`.
- Questa logica si generalizza facilmente a N numeri usando un ciclo, che vedremo negli esercizi successivi.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero1, numero2, numero3;

    printf("Inserire il primo numero: ");
    scanf("%d", &numero1);
    printf("Inserire il secondo numero: ");
    scanf("%d", &numero2);
    printf("Inserire il terzo numero: ");
    scanf("%d", &numero3);

    // Verifico quale sia il maggiore fra i primi due numeri
    // quindi confronto il maggiore con il terzo
    int maggiore;
    if(numero1 > numero2)
        maggiore = numero1;
    else
        maggiore = numero2;
    if(numero3 > maggiore)
        maggiore = numero3;

    printf("Il maggiore è %d!\n", maggiore);

    return 0;
}
```


2025 Daniele Prevedello

4.3.9 Maggiore fra tre numeri (versione 3)

Dati tre numeri inseriti dall'utente, il programma stampa il maggiore. In questa versione si utilizza l'operatore ternario `?:` (if-inline).

Obiettivo

Leggere tre numeri da tastiera e visualizzare il valore più alto tra i tre, sfruttando l'operatore condizionale `?:`.

Descrizione

In questa terza versione si introduce l'**operatore ternario** `?:`, detto anche *if-inline*, che permette di scrivere un'espressione condizionale in forma compatta:

Text Only

```
condizione ? valore_se_vero : valore_se_falso
```

L'operatore può essere usato per assegnare il risultato parziale a una variabile d'appoggio `maggiore`, oppure direttamente all'interno dell'istruzione `printf`, rendendo il codice molto conciso.

Ad esempio, per trovare il maggiore tra `a` e `b`:

C

```c
int maggiore = (a > b) ? a : b;
```

Suggerimenti

- L'operatore `?:` è utile per espressioni semplici, ma non abusarne: quando la logica è complessa, un `if-else` esplicito è più leggibile.
- Puoi annidare due operatori ternari per coprire tutti e tre i valori, oppure usare una variabile d'appoggio intermedia.
- Confronta questa versione con le precedenti: stessa logica, sintassi diversa.
- L'operatore ternario ha una precedenza bassa; usa le parentesi per evitare ambiguità.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero1, numero2, numero3;

    printf("Inserire il primo numero: ");
    scanf("%d", &numero1);
    printf("Inserire il secondo numero: ");
    scanf("%d", &numero2);
    printf("Inserire il terzo numero: ");
    scanf("%d", &numero3);

    // Verifico quale sia il maggiore fra i primi due numeri
    // quindi confronto il maggiore con il terzo
    int maggiore = (numero1 > numero2 ? numero1 : numero2);
    printf("Il maggiore è %d!\n", (numero3 > maggiore ? numero3 : maggiore));

    return 0;
}
```


2025 Daniele Prevedello

4.3.10 Numero divisibile per 2 o 3

Dato un numero intero, il programma verifica se è divisibile per 2 o per 3 e, in base al risultato, calcola il triplo oppure la metà del numero.

Obiettivo

Leggere un numero intero da tastiera e applicare la seguente regola: se è divisibile per 2 o per 3, stamparne il triplo; altrimenti stamparne la metà.

Descrizione

L'esercizio combina l'uso dell'operatore modulo `%` con un operatore logico composto. Un numero è divisibile per 2 se `n % 2 == 0`, divisibile per 3 se `n % 3 == 0`. La condizione composta si scrive con l'operatore logico **OR** (`||`):

Text Only

```
1       if (n % 2 == 0 || n % 3 == 0)
```

Se la condizione è vera si calcola il triplo (`3 * n`), altrimenti si calcola la metà (`n / 2.0`).

Suggerimenti

- Usa `n / 2.0` (con il punto decimale) per ottenere un risultato in virgola mobile nel caso la metà non sia intera; in alternativa dichiara `n` come `float` o `double`.
- Ricorda la differenza tra `||` (OR logico, vera se almeno una condizione è vera) e `&&` (AND logico, vera solo se entrambe le condizioni sono vere).
- Testa il programma con numeri divisibili solo per 2 (es. 4), solo per 3 (es. 9), per entrambi (es. 6) e per nessuno dei due (es. 5).
- Nota che il numero 0 è divisibile per qualsiasi numero: verifica come si comporta il tuo programma con `n = 0`.

Soluzione

C

```c
1       #include <stdio.h>
2
3       int main(void) {
4           int numero;
5
6           printf("Inserire un numero: ");
7           scanf("%d", &numero);
8
9           // Verifico che il numero sia divisibile per due o per tre.
10          // Per verificare la divisibilità si usa l'operatore %, per
11          // la congiunzione "oppure" si usa l'operatore or ||
12          if(numero%2 == 0 || numero%3 == 0)
13              printf("Il triplo è: %d\n", numero*3);
14          else
15              printf("La metà è: %.1f\n", numero/2.0);
16
17          return 0;
18      }
```


2025 Daniele Prevedello

4.3.11 Numero divisibile per 3 e 7

Dato un numero intero, il programma verifica se è divisibile sia per 3 che per 7, comunicando l'esito sia in caso positivo che negativo.

Obiettivo

Leggere un numero intero da tastiera e stabilire se è divisibile contemporaneamente per 3 e per 7, stampando un messaggio in entrambi i casi.

Descrizione

A differenza dell'esercizio precedente (dove bastava la divisibilità per uno dei due), qui entrambe le condizioni devono essere soddisfatte contemporaneamente. Si usa l'operatore logico **AND** (`&&`):

Text Only

```
if (n % 3 == 0 && n % 7 == 0)
```

In alternativa, si può sfruttare il fatto che un numero divisibile sia per 3 che per 7 è anche divisibile per 21 (il loro minimo comune multiplo, poiché 3 e 7 sono coprimi):

Text Only

```
if (n % 21 == 0)
```

Suggerimenti

- Usa l'operatore `&&` per verificare che entrambe le condizioni siano vere allo stesso tempo.
- Considera l'equivalenza con `n % 21 == 0` e rifletti sul perché è valida (3 e 7 sono numeri primi, quindi il loro mcm è $3 \times 7 = 21$).
- Testa il programma con i valori 21, 42, 3, 7, 1 e 0 per coprire i casi più significativi.
- Assicurati di stampare un messaggio anche quando la condizione non è soddisfatta (ramo `else`).

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    printf("Inserire un numero: ");
    scanf("%d", &numero);

    // Verifico che il numero sia divisibile per tre e per sette.
    // Per verificare la divisibilità si usa l'operatore %, per la
    // congiunzione "e" si usa l'operatore and &&
    if(numero%3 == 0 && numero%7 == 0)
        printf("Il numero è divisibile per 3 e 7\n");
    else
        printf("Il numero NON è divisibile per 3 e 7\n");

    return 0;
}
```


2025 Daniele Prevedello

4.3.12 Test anno bisestile

Il programma chiede all'utente un numero intero che rappresenta un anno e verifica se esso è bisestile.

Obiettivo

Leggere un anno da tastiera e stabilire se è bisestile, applicando la regola del calendario gregoriano.

Descrizione

Nel calendario gregoriano, un anno è **bisestile** se rispetta le seguenti regole:

1. È divisibile per 4, **e**
2. Non è divisibile per 100, **oppure** è divisibile per 400.

In forma di espressione logica:

Text Only

```
(anno % 4 == 0 && anno % 100 != 0) || (anno % 400 == 0)
```

Esempi:

- 2024 è bisestile (divisibile per 4, non per 100)
- 1900 **non** è bisestile (divisibile per 100, ma non per 400)
- 2000 è bisestile (divisibile per 400)

Suggerimenti

- Scrivi la condizione usando gli operatori `&&`, `||` e `!=` come mostrato sopra, facendo attenzione alle parentesi per rispettare la precedenza degli operatori.
- In alternativa, puoi usare `if-else` annidati: prima controlla la divisibilità per 400, poi per 100, infine per 4.
- Testa il programma con gli anni 2000, 1900, 2024 e 2023 per verificare tutti i casi.
- Gli anni negativi non esistono nel calendario gregoriano: potresti aggiungere un controllo sull'input.

Soluzione

C

```c
#include <stdio.h>

int main(){
    int anno;

    printf("Inserire un anno: ");
    scanf("%d", &anno);

    if( (anno%4 == 0 && anno%100 != 0) || anno%400 == 0 )
        printf("\nIl %d è un anno bisestile!\n", anno);
    else
        printf("\nIl %d NON è un anno bisestile!\n", anno);

    return 0;
}
```


2025 Daniele Prevedello

4.3.13 Conversione tempo in orario

Il programma riceve dall'utente un tempo espresso in secondi e lo converte nel formato ore, minuti e secondi (`hh:mm:ss`).

Obiettivo

Leggere un numero intero di secondi da tastiera e stampare il corrispondente orario nel formato `hh:mm:ss`.

Descrizione

Per convertire un totale di secondi nel formato orario si sfruttano la divisione intera (`/`) e l'operatore modulo (`%`):

- **Ore**: `secondi / 3600`
- **Minuti**: `(secondi % 3600) / 60`
- **Secondi rimanenti**: `secondi % 60`

Ad esempio, 3723 secondi corrispondono a 1 ora, 2 minuti e 3 secondi (01:02:03).

Suggerimenti

- Usa la divisione intera `/` e l'operatore modulo `%` per estrarre le tre componenti.
- Per la stampa nel formato `hh:mm:ss` usa il formato `%02d` in `printf`: il `2` indica la larghezza minima e lo `0` riempie con zeri a sinistra (es. `5` diventa `05`).
- Verifica che il valore in ingresso sia non negativo: un numero di secondi negativo non ha senso fisico.
- Testa con i valori 0, 59, 60, 3600 e 86399 (23:59:59, l'ultimo secondo della giornata).

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int secondi;
    printf("Inserire un numero di secondi: ");
    scanf("%d", &secondi);

    if( secondi<0 )
        printf("Valore inserito non corretto.\n");
    else {
        int hh = secondi/3600;
        int mm = (secondi - hh*3600)/60;
        int ss = secondi - hh*3600 - mm*60;
        printf("Orario: %dh:%dm:%ds\n", hh, mm, ss);
    }

    return 0;
}
```


2025 Daniele Prevedello

4.3.14 Convertitore di temperature

Dato un valore di temperatura in gradi Celsius, il programma lo converte in Fahrenheit e in Kelvin, segnalando un errore se il valore è inferiore allo zero assoluto.

Obiettivo

Acquisire una temperatura in gradi Celsius e stamparle le corrispondenti temperature in gradi Fahrenheit e in Kelvin. Se il valore inserito è inferiore allo zero assoluto, visualizzare un messaggio di errore.

Descrizione

Ricordiamo che:

\(Fahrenheit = \frac{9}{5} \cdot Celsius + 32\)

\(Kelvin = Celsius + 273{,}15\)

Lo zero assoluto corrisponde a \(-273{,}15\,°C\): al di sotto di questo valore la temperatura non ha significato fisico.

Suggerimenti

- Utilizza una variabile `double` per la temperatura, poiché i valori possono essere decimali.
- Controlla prima se il valore è valido (maggiore o uguale a \(-273{,}15\)) e poi esegui le conversioni.
- Ricorda che la divisione `9/5` tra interi in C dà `1`: scrivi `9.0/5.0` oppure usa la costante `1.8`.
- Usa `printf` con il formato `%.2f` per visualizzare i risultati con due cifre decimali.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    float celsius;

    printf("Inserire la temperatura in Celsius: ");
    scanf("%f", &celsius);

    if(celsius < -273.15)
        printf("\nErrore: la temperatura è inferiore allo zero assoluto (-273,15 °C)\n");
    else {
        float fahrenheit = (9./5) * celsius + 32;
        float kelvin = celsius + 273.15 ;

        printf("\nLa temperatura in Fahrenheit è di %.2f°F\n", fahrenheit);
        printf("La temperatura in Kelvin è di %.2f°K\n", kelvin);
    }

    return 0;
}
```


2025 Daniele Prevedello

4.3.15 Stampa dei mesi

Dato un numero intero tra 1 e 12, il programma stampa per esteso il nome del mese corrispondente.

Obiettivo

Acquisire un numero intero che rappresenta un mese e visualizzare il nome del mese corrispondente (da "Gennaio" a "Dicembre"). Se il numero non è compreso tra 1 e 12, segnalare un errore.

Descrizione

Questo esercizio è un'applicazione classica del costrutto `switch-case`: a ciascun valore intero viene associata un'azione specifica (la stampa del nome del mese). Il caso `default` gestisce i valori non validi.

Suggerimenti

- Usa il costrutto `switch` con un `case` per ciascuno dei 12 mesi.
- Non dimenticare l'istruzione `break` al termine di ogni `case`, altrimenti l'esecuzione prosegue nel caso successivo.
- Aggiungi un `case default` per gestire valori fuori dall'intervallo 1–12.
- In alternativa allo `switch`, potresti risolvere l'esercizio con una catena di `if-else if`, ma lo `switch` risulta più leggibile in questo caso.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int main(void) {
    int mese;

    printf("Inserire il mese corrente: ");
    scanf("%d", &mese);

    // Viene scelto il mese da stampare tramite il
    // costrutto switch-case
    switch(mese){
        case 1:
            printf("Il mese è Gennaio\n");
            break;
        case 2:
            printf("Il mese è Febbraio\n");
            break;
        case 3:
            printf("Il mese è Marzo\n");
            break;
        case 4:
            printf("Il mese è Aprile\n");
            break;
        case 5:
            printf("Il mese è Maggio\n");
            break;
        case 6:
            printf("Il mese è Giugno\n");
            break;
        case 7:
            printf("Il mese è Luglio\n");
            break;
        case 8:
            printf("Il mese è Agosto\n");
            break;
        case 9:
            printf("Il mese è Settembre\n");
            break;
        case 10:
            printf("Il mese è Ottobre\n");
            break;
        case 11:
            printf("Il mese è Novembre\n");
            break;
        case 12:
            printf("Il mese è Dicembre\n");
            break;
        default:
            printf("Il numero inserito non corrisponde ad un mese!\n");
    };

    return 0;
}
```


2025 Daniele Prevedello

4.3.16 Semplice calcolatrice

Il programma realizza una calcolatrice che esegue una delle quattro operazioni fondamentali scelta dall'utente.

Obiettivo

Acquisire due operandi e un operatore (`+`, `-`, `*`, `/`) e stampare il risultato dell'operazione selezionata. Gestire il caso di divisione per zero.

Descrizione

L'utente inserisce due numeri e un carattere che rappresenta l'operazione da eseguire. Il programma seleziona l'operazione tramite uno `switch` sul carattere inserito e ne stampa il risultato. Per la divisione è necessario verificare che il secondo operando non sia zero.

Suggerimenti

- Acquisisci l'operatore come variabile di tipo `char`.
- Usa `switch` sul carattere dell'operatore: i `case` saranno `'+'`, `'-'`, `'*'`, `'/'`.
- Prima di eseguire la divisione, verifica che il divisore sia diverso da zero.
- Usa `double` per gli operandi in modo da gestire correttamente la divisione tra numeri non interi.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    float numero1, numero2;
    int operazione;

    printf("Inserire il primo operando: ");
    scanf("%f", &numero1);
    printf("Inserire il secondo operando: ");
    scanf("%f", &numero2);

    printf("\nQuale operazione vuoi eseguire?\n");
    printf("1) somma\n");
    printf("2) sottrazione\n");
    printf("3) moltiplicazione\n");
    printf("4) divisione\n");
    printf("Scelta: ");
    scanf("%d", &operazione);

    switch( operazione ){
        case 1:
            printf("\n%f + %f = %f\n", numero1, numero2, numero1+numero2);
            break;
        case 2:
            printf("\n%f - %f = %f\n", numero1, numero2, numero1-numero2);
            break;
        case 3:
            printf("\n%f * %f = %f\n", numero1, numero2, numero1*numero2);
            break;
        case 4:
            printf("\n%f / %f = %f\n", numero1, numero2, numero1/numero2);
            break;
        default:
            printf("\nLa scelta non è corretta!\n");
    };

    return 0;
}
```


2025 Daniele Prevedello

4.4 Cicli

4.4.1 Somma dei primi 12 numeri naturali

Il programma calcola la somma dei primi 12 numeri naturali (da 1 a 12) utilizzando un ciclo `for`.

Obiettivo

Calcolare e stampare il valore di $1 + 2 + 3 + \ldots + 12$ tramite un ciclo.

Descrizione

Questo esercizio introduce l'uso del ciclo `for` per accumulare una somma. Si parte da una variabile `somma` inizializzata a 0 e si aggiunge il valore del contatore a ogni iterazione.

Il risultato atteso è 78, ottenibile anche con la formula di Gauss: $\frac{n(n+1)}{2} = \frac{12 \cdot 13}{2} = 78$.

Suggerimenti

- Dichiara una variabile `int somma = 0` prima del ciclo.
- Il ciclo `for` deve iterare con `i` che va da 1 a 12 incluso.
- All'interno del ciclo, aggiorna la somma con `somma += i`.
- Dopo il ciclo, stampa il risultato con `printf`.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    // Ovviamente questa è una soluzione, ma non la useremo.
    // int risultato = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12;

    int risultato = 0;
    for(int n = 1; n<=12; n++)
        risultato += n;

    printf("La somma dei primi 12 numeri vale: %d\n", risultato);
    return 0;
}
```


2025 Daniele Prevedello

4.4.2 Somma numeri da 10 a 20

Il programma calcola la somma di tutti i numeri interi compresi tra 10 e 20 (estremi inclusi).

Obiettivo

Calcolare e stampare il valore di $10 + 11 + 12 + \dots + 20$ tramite un ciclo `for`.

Descrizione

Questo esercizio è una variante del precedente: il ciclo non parte da 1 ma da un valore diverso da zero. Il risultato atteso è 165.

Suggerimenti

- Dichiara una variabile `int somma = 0` prima del ciclo.
- Il ciclo `for` deve iterare con `i` che parte da 10 e arriva a 20 incluso.
- La condizione di uscita è `i <= 20`.
- Prova a verificare il risultato con la formula di Gauss applicata all'intervallo: $\frac{(10 + 20) \cdot 11}{2} = 165$.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int risultato = 0;
    for(int n = 10; n<=20; n++)
        risultato += n;

    printf("La somma dei numeri da 10 a 20 vale: %d\n", risultato);
    return 0;
}
```


2025 Daniele Prevedello

4.4.3 Vari conteggi con il `for`

Il programma esegue sei sequenze di stampa diverse utilizzando altrettanti cicli `for`, esplorando le principali varianti del costrutto.

Obiettivo

Stampare in sequenza:

1. I numeri da 1 a 100 (incremento di 1)
2. I numeri da 100 a 1 (decremento di 1)
3. I numeri pari da 1 a 100
4. I numeri dispari da 100 a 1
5. La tabellina del 3 fino a 100
6. Le potenze del 2 fino a 10000

Descrizione

Questo esercizio mostra come il ciclo `for` sia flessibile: può contare in avanti, all'indietro, a passi diversi da 1 o con aggiornamenti moltiplicativi. Ogni sequenza richiede di scegliere opportunamente il valore iniziale, la condizione di uscita e l'istruzione di aggiornamento.

Suggerimenti

- Per i numeri dispari da 100 a 1, parti da 99 (primo dispari sotto 100) e decrementa di 2.
- Per la tabellina del 3, usa `i += 3` oppure stampa `3 * i` con `i` che va da 1 in poi.
- Per le potenze del 2, l'aggiornamento è `i *= 2` anziché `i++`.
- Separa le sei sequenze con una riga vuota o un'intestazione per rendere l'output più leggibile.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    printf("Numeri da 1 a 100\n");
    for(int n=1; n<=100; n++)
        printf("%d ", n);

    printf("\n\nNumeri da 100 a 1\n");
    for(int n=100; n>=1; n--)
        printf("%d ", n);

    printf("\n\nNumeri da 1 a 100 pari\n");
    for(int n=2; n<=100; n+=2)
        printf("%d ", n);

    printf("\n\nNumeri da 100 a 1 dispari\n");
    for(int n=99; n>=1; n-=2)
        printf("%d ", n);

    printf("\n\nLa tabellina del 3 fino a 100\n");
    for(int n=1; n*3<=100; n++)
        printf("%d ", n*3);

    printf("\n\nPotenze del 2 minori di 10000\n");
    for(int n=1; n<=10000; n*=2)
        printf("%d ", n);

    printf("\n\n");
    return 0;
}
```


2025 Daniele Prevedello

4.4.4 Indovina il numero

Il programma genera un numero casuale tra 1 e 100 e sfida l'utente a indovinarlo in massimo 10 tentativi, fornendo un suggerimento dopo ogni risposta errata.

Obiettivo

Generare un numero segreto casuale compreso tra 1 e 100. A ogni tentativo dell'utente, comunicare se il numero segreto è maggiore o minore di quello inserito. Il gioco termina quando l'utente indovina oppure esaurisce i 10 tentativi.

Descrizione

Il numero casuale viene generato con la funzione `rand()` della libreria `<stdlib.h>`. Per ottenere sequenze diverse a ogni esecuzione, è necessario inizializzare il generatore con `srand(time(NULL))`, usando la libreria `<time.h>`.

Il ciclo `for` gestisce il numero massimo di tentativi; un'istruzione `break` permette di uscire dal ciclo in anticipo se l'utente indovina.

Suggerimenti

- Includi `<stdlib.h>` e `<time.h>` per `rand()`, `srand()` e `time()`.
- Per generare un numero tra 1 e 100: `numero_segreto = rand() % 100 + 1`.
- Usa una variabile booleana (o un intero `indovinato`) per sapere se uscire con successo.
- Dopo il ciclo, verifica se l'utente ha vinto o ha esaurito i tentativi e stampa il messaggio appropriato.

Soluzione

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));
    int n = rand() % 100 + 1;

    int valore=0;
    printf("Indovina il numero! (tra 1 e 100)\n");
    for(int i=1; i<=10 && valore!=n; i++){
        printf("Tentativo %d: ", i);
        scanf("%d", &valore);

        if(valore == n)
            printf("Complimenti hai indovinato in %d mosse!\n", i);
        else if(valore > n)
            printf("%d è troppo grande... riprova!\n", valore);
        else
            printf("%d è troppo piccolo... riprova!\n", valore);
    }
    return 0;
}
```


2025 Daniele Prevedello

4.4.5 Numero di cifre di un intero

Il programma calcola il numero di cifre di un intero letto in input, sia in base 10 sia in base 2.

Obiettivo

Dato un numero intero letto da tastiera, determinare:

1. Quante cifre ha in base 10 (sistema decimale).
2. Quante cifre ha in base 2 (sistema binario).

Descrizione

Il numero di cifre decimali di un intero positivo \(n\) si può calcolare contando quante volte si riesce a dividere \(n\) per 10 prima di ottenere 0. La stessa logica vale in base 2, dividendo per 2.

Ad esempio, il numero \(345\) ha 3 cifre decimali e, poiché \(345 = 101011001_2\), ha 9 cifre binarie.

Suggerimenti

- Usa un ciclo `for` (o `while`) che divide ripetutamente il numero per la base (10 o 2) e conta le iterazioni.
- Attenzione al caso `n = 0`: ha 1 cifra in entrambe le basi; gestiscilo separatamente con un `if`.
- Se il numero può essere negativo, lavora sul valore assoluto prima di contare le cifre.
- In alternativa, per la base 10, potresti usare `floor(log10(n)) + 1` dalla libreria `<math.h>`, ma il metodo iterativo è più diretto.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    printf("Inserire un numero: ");
    scanf("%d", &numero);

    int cifre = 1;
    for(int base=10; numero/base > 0; base*=10)
        cifre++;

    printf("%d è composto da %d cifr%c in base 10\n", numero, cifre, cifre==1 ? 'a':'e');

    cifre = 1;
    for(int base=2; numero/base > 0; base*=2)
        cifre++;

    printf("%d è composto da %d cifr%c in base 2\n", numero, cifre, cifre==1 ? 'a':'e');

    return 0;
}
```


2025 Daniele Prevedello

4.4.6 Massimo e media non definiti a priori

Il programma acquisisce una sequenza di numeri di lunghezza non nota in anticipo e ne calcola il massimo e la media.

Obiettivo

Chiedere all'utente di inserire un numero qualsiasi di valori (la sequenza termina con un valore sentinella concordato, ad esempio `0` o un valore speciale). Al termine, stampare il massimo e la media dei valori inseriti.

Descrizione

A differenza degli esercizi precedenti, qui il numero di elementi non è fissato: si usa un ciclo `do-while` (o `while`) che continua finché l'utente non decide di fermarsi. È necessario tenere traccia del numero di valori inseriti per calcolare la media.

Il massimo viene aggiornato a ogni iterazione confrontando il valore corrente con il massimo temporaneo.

Suggerimenti

- Inizializza il massimo con il primo valore letto, non con un valore arbitrario come `0` (che potrebbe essere maggiore di tutti i valori negativi inseriti).
- Conta gli elementi inseriti con un contatore `n` per calcolare la media come `somma / n`.
- Usa un ciclo `do-while` se vuoi garantire almeno un'iterazione, oppure un `while` con una condizione sull'input sentinella.
- Verifica il caso in cui l'utente non inserisca nessun valore valido (n = 0) per evitare divisioni per zero.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int valore, max, somma=0;
    int inseriti = 0;
    // int uscita;
    char uscita;

    do {
        printf("Inserire un numero: ");
        scanf("%d", &valore);

        if( inseriti == 0 )
            max = valore;
        else if( valore > max )
            max = valore;
        somma += valore;
        inseriti++;

        printf("Vuoi terminare (S o N)? ");
        // Uso \n%c per evitare la lettura dello \n lasciato dalla precedente scanf
        scanf("\n%c", &uscita);
        printf("\n");
    } while( uscita != 's' && uscita != 'S' );

    printf("Il valore massimo è %d\n", max);
    printf("Il valor medio è: %.2f\n", (float)somma/inseriti);

    return 0;
}
```


2025 Daniele Prevedello

4.4.7 Generazione numeri casuali

Il programma genera un numero casuale per ciascuno dei sei intervalli specificati e ne stampa il valore.

Obiettivo

Generare e stampare un numero casuale per ciascuno dei seguenti intervalli:

1. \([0,\ 100)\) — intero, 0 incluso, 100 escluso
2. \([10,\ 100)\) — intero, 10 incluso, 100 escluso
3. \([10,\ 100]\) — intero, 10 incluso, 100 incluso
4. \([-50,\ 100)\) — intero, -50 incluso, 100 escluso
5. \([0{,}00,\ 1{,}00]\) — numero reale tra 0 e 1
6. \([10\,000,\ 100\,000]\) — numero reale tra 10 000 e 100 000

Descrizione

La funzione `rand()` della libreria `<stdlib.h>` restituisce un intero pseudo-casuale nell'intervallo \([0,\ RAND_MAX]\). Per ottenere numeri in un intervallo arbitrario si usa l'operatore modulo `%` e uno scostamento. Per i numeri reali si divide il risultato di `rand()` per `(double)RAND_MAX`.

La formula generale per un intero in \([a,\ b]\) è:

\(n = rand() \% (b - a + 1) + a \)

Suggerimenti

- Includi `<stdlib.h>` e `<time.h>`, e chiama `srand(time(NULL))` all'inizio del `main` per avere sequenze diverse a ogni esecuzione.
- Per l'intervallo semiaperto \([a,\ b)\), usa `rand() % (b - a) + a`.
- Per un reale in \([0,\ 1]\), usa `(double)rand() / RAND_MAX`.
- Per scalare il reale in \([min,\ max]\), usa `min + (double)rand() / RAND_MAX * (max - min)`.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>
#include <stdlib.h>     // Libreria inclusa per rand() e srand()
#include <time.h>       // Libreria inclusa per time()

int main(void) {
    int numero1 = rand();
    int numero2 = rand();
    int numero3 = rand();
    printf("Generazione numeri tra 0 e RAND_MAX (%d)\n", RAND_MAX);
    printf("Da notare che se non si invoca srand(), i numeri rimangono\n");
    printf("gli stessi fra successive esecuzioni:\n");
    printf("%d  %d  %d\n\n", numero1, numero2, numero3);

    srand(15);
    int numero4 = rand();
    int numero5 = rand();
    int numero6 = rand();
    printf("Anche se viene messo un parametro fisso (15 nell'esempio) in srand(),\n");
    printf("i numeri rimangono gli stessi fra successive esecuzioni:\n");
    printf("%d  %d  %d\n\n", numero4, numero5, numero6);

    srand( time(NULL) );    // Inizializzazione del seed della rand ad un valore legato al tempo corrente
    int numero7 = rand();
    int numero8 = rand();
    int numero9 = rand();
    printf("Se si parametrizza srand() con un valore legato al tempo corrente invece,\n");
    printf("i numeri visualizzati sranno sempre differenti:\n");
    printf("%d  %d  %d\n\n", numero7, numero8, numero9);

    // Generazione di un numero tra [0 e 100)    Zero incluso, cento escluso
    int numero10 = rand() % 100;
    printf("Numero tra [0 e 100): %d\n", numero10);

    // Generazione di un numero tra [10 e 100)    Dieci incluso, cento escluso
    int numero11 = 10 + rand() % (100 - 10);  // o semplicemente: 10 + rand() % 90
    printf("Numero tra [10 e 100): %d\n", numero11);

    // Generazione di un numero tra [10 e 100]    Dieci incluso, cento incluso
    int numero12 = 10 + rand() % (100 + 1 - 10);  // o semplicemente: 10 + rand() % 91
    printf("Numero tra [10 e 100]: %d\n", numero12);

    // Generazione di un numero tra [-50 e 100)    meno cinquanta incluso, cento incluso
    int numero13 = -50 + rand() % (100 - -50);  // o semplicemente: -50 + rand() % 150
    printf("Numero tra [-50 e 100): %d\n", numero13);

    // Generazione di un numero tra [0,00 e 1,00]    zezo incluso, uno incluso con due cifre decimali
    float numero14 = (rand() % (100 + 1)) / 100.0;  // o semplicemente: (rand() % 101) / 100.0
    printf("Numero tra [0.00 e 1.00] con due cifre decimali: %.2f\n", numero14);

    // Generazione di un numero tra [10,000 e 100,000]    dieci incluso, cento incluso con tre cifre decimali
    float numero15 = (rand() % (1000 + 1)) / 1000.0;  // o semplicemente: (rand() % 1001) / 1000.0
    numero15 = 10 + rand() % (100 - 10) + numero15;
    printf("Numero tra [10,000 e 100,000] con tre cifre decimali: %.3f\n", numero15);

    return 0;
}
```


2025 Daniele Prevedello

4.4.8 Operazioni su numeri casuali

Generare 100 numeri casuali tra 0 e 1000 e calcolare minimo, massimo, media e somma dei valori ottenuti.

Obiettivo

Il programma deve generare automaticamente 100 numeri interi casuali compresi tra 0 e 1000, quindi calcolare e stampare: il valore minimo, il valore massimo, la somma totale e la media aritmetica dei numeri generati.

Descrizione

Per generare numeri casuali in C si utilizzano le funzioni `rand()` e `srand()` della libreria `<stdlib.h>`. È buona prassi inizializzare il generatore con `srand(time(NULL))` (dalla libreria `<time.h>`) in modo che la sequenza sia diversa ad ogni esecuzione.

Il minimo e il massimo si trovano scorrendo il vettore e aggiornando le variabili di appoggio ogni volta che si trova un valore più piccolo o più grande. La media si ottiene dividendo la somma totale per il numero di elementi.

Suggerimenti

- Inizializza il generatore di numeri casuali con `srand(time(NULL))` prima del ciclo, altrimenti otterrai sempre la stessa sequenza.
- Per ottenere un numero casuale nell'intervallo [0, 1000] usa la formula `rand() % 1001`.
- Inizializza `min` e `max` con il primo elemento generato, non con valori arbitrari come 0 o 9999.
- Usa un unico ciclo `for` per generare i numeri e aggiornare contemporaneamente somma, minimo e massimo.

Soluzione

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));

    int numero, minimo, massimo, somma=0;

    for(int i=0; i<100; i++){
        numero = rand() % 1001;
        printf("%d ", numero);

        if(i==0)
            minimo = massimo = numero;
        else {
            if(numero < minimo)
                minimo = numero;
            if(numero > massimo)
                massimo = numero;
        }
        somma += numero;
    }

    printf("\n\nIl minimo è: %d\n", minimo);
    printf("Il massimo è: %d\n", massimo);
    printf("La media vale: %.3f\n", somma/100.0);
    printf("La somma vale: %d\n", somma);

    return 0;
}
```


2025 Daniele Prevedello

4.4.9 Tavola pitagorica

Stampare la tavola pitagorica completa, con una griglia che renda leggibili righe e colonne.

Obiettivo

Il programma deve visualizzare a schermo la classica tavola pitagorica (da 1×1 a 10×10), formattata con una griglia che allinei correttamente i valori.

Descrizione

La tavola pitagorica è una tabella in cui ogni cella alla riga i e colonna j contiene il prodotto i × j. Per stamparla in modo leggibile è necessario usare due cicli `for` annidati: il ciclo esterno scorre le righe, quello interno scorre le colonne.

La formattazione è importante: usando `printf` con una larghezza fissa (ad esempio `%4d`) i numeri risultano allineati e la griglia è più chiara. Si può aggiungere una riga di intestazione con i numeri da 1 a 10 e una linea separatrice per migliorare la leggibilità.

Suggerimenti

- Usa due cicli `for` annidati: il primo per le righe (i da 1 a 10), il secondo per le colonne (j da 1 a 10).
- Stampa ogni prodotto con `printf("%4d", i * j)` per mantenere l'allineamento.
- Aggiungi una riga di intestazione con i valori da 1 a 10 e una linea di separazione (es. con trattini) prima del corpo della tabella.
- Al termine di ogni riga ricordati di andare a capo con `printf("\n")`.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    printf("\n-------------------------------------------------\n");
    for(int r=1; r<=10; r++){
        for(int c=1; c<=10; c++)
            if( r*c < 100 )
                printf("| %2d ", r*c);
            else
                printf("| %d", r*c);
        printf("|\n-------------------------------------------------\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.4.10 Stampa rettangolo base

Stampare un rettangolo composto dal carattere `#`, con dimensioni fisse di 10 righe e 20 colonne.

Obiettivo

Il programma deve disegnare su schermo un rettangolo pieno di 10 righe e 20 colonne, usando il carattere `#` per riempire tutte le celle.

Descrizione

Il rettangolo è una figura semplice da realizzare con due cicli `for` annidati: il ciclo esterno gestisce le righe, quello interno stampa i caratteri di ogni riga. Poiché il rettangolo è pieno, ogni cella contiene il carattere `#` senza distinzione tra bordo e interno.

Questo esercizio costituisce la base per varianti più complesse in cui si distinguono bordo e contenuto oppure si usano dimensioni variabili.

Suggerimenti

- Usa due cicli `for` annidati: `for(int i = 0; i < 10; i++)` per le righe e `for(int j = 0; j < 20; j++)` per le colonne.
- All'interno del ciclo interno stampa il carattere `#` senza andare a capo.
- Al termine di ogni riga (dopo il ciclo interno) stampa un `\n` per passare alla riga successiva.
- Le costanti 10 e 20 possono essere definite come macro con `#define RIGHE 10` e `#define COLONNE 20` per rendere il codice più leggibile.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    for(int r=1; r<=10; r++){
        for(int c=1; c<=20; c++)
            if( r==1 || r==10 || c==1 || c==20 )
                printf("#");
            else
                printf(" ");
        printf("\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.4.11 Stampa rettangolo personalizzato

Stampare un rettangolo con caratteri scelti dall'utente per bordo e contenuto, con dimensioni minime di 3 righe e 3 colonne.

Obiettivo

Il programma deve chiedere all'utente: il numero di righe e colonne (entrambi almeno 3), il carattere per il bordo e il carattere per il contenuto interno. Deve quindi disegnare il rettangolo rispettando queste impostazioni.

Descrizione

A differenza del rettangolo base, qui è necessario distinguere tra le celle di bordo e quelle interne. Una cella appartiene al bordo se si trova sulla prima riga, sull'ultima riga, sulla prima colonna o sull'ultima colonna; tutte le altre celle sono interne.

La validazione dell'input con un ciclo `do-while` garantisce che l'utente inserisca valori ammissibili (almeno 3) prima di procedere con il disegno.

Suggerimenti

- Usa un ciclo `do-while` per chiedere righe e colonne, ripetendo la richiesta finché i valori inseriti sono inferiori a 3.
- All'interno del doppio ciclo `for`, usa una condizione `if-else` per decidere quale carattere stampare: bordo se `i == 0 || i == righe-1 || j == 0 || j == colonne-1`, altrimenti contenuto.
- Leggi i caratteri con `scanf(" %c", &c)` (nota lo spazio prima di `%c`) per evitare che vengano letti caratteri bianchi residui nel buffer.
- Testa il programma con dimensioni minime (3×3) per verificare che il bordo e il contenuto siano corretti.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    char bordo, contenuto;
    int righe, colonne;

    printf("Quale carattere si vuole usare per il bordo? ");
    scanf("%c", &bordo);
    printf("Quale carattere si vuole usare per il contenuto? ");
    scanf("\n%c", &contenuto);

    do{
        printf("\nQuante righe vuoi (minimo 3): ");
        scanf("%d", &righe);
        if( righe<3 )
            printf("Le righe devono essere almeno 3!");
    }while( righe<3 );
    do{
        printf("\nQuante colonne vuoi (minimo 3): ");
        scanf("%d", &colonne);
        if( colonne<3 )
            printf("Le colonne devono essere almeno 3!");
    }while( colonne<3 );

    for(int r=1; r<=righe; r++){
        for(int c=1; c<=colonne; c++)
            if( r==1 || r==righe || c==1 || c==colonne )
                printf("%c", bordo);
            else
                printf("%c", contenuto);
        printf("\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.4.12 Numeri casuali e rettangoli

Generare due numeri casuali R e C tra 5 e 20 e disegnare un rettangolo le cui caratteristiche dipendono dai valori estratti.

Obiettivo

Il programma deve generare casualmente il numero di righe R e di colonne C (entrambi tra 5 e 20), quindi disegnare un rettangolo seguendo queste regole:

- il rettangolo è **pieno** se R è pari, **vuoto** (solo bordo) se R è dispari;
- il carattere usato è `*` se C è minore di 10, oppure `#` se C è maggiore o uguale a 10.

Descrizione

Questo esercizio combina la generazione di numeri casuali con il disegno di figure geometriche condizionale. Le dimensioni e l'aspetto del rettangolo cambiano ad ogni esecuzione, rendendo il programma dinamico.

Per determinare se il rettangolo è pieno o vuoto, si usa la parità di R. Per distinguere bordo e interno nelle versioni vuote, si applica la stessa logica dell'esercizio precedente: una cella è di bordo se si trova sulla prima o ultima riga oppure sulla prima o ultima colonna.

Suggerimenti

- Per generare R e C nell'intervallo [5, 20] usa la formula `rand() % 16 + 5`.
- Stampa i valori di R e C generati, così l'utente può vedere da cosa dipende il disegno.
- Usa una variabile `char c` per memorizzare il carattere scelto in base a C, prima di entrare nei cicli.
- Gestisci i due casi (pieno/vuoto) con una condizione `if-else` all'interno del doppio ciclo `for`.

Soluzione

C

```c
/*******************************************************************************

Generare due numeri casuali R e C tra 5 e 20, e disegnare su schermo un rettangolo
di R righe e C colonne, pieno se R è pari, vuoto se dispari.
Il carattere che comporrà il rettangolo (sia vuoto che pieno) sarà un * se C < 10
oppure un # se C è >= 10.

*******************************************************************************/

#include <stdio.h>
#include <time.h>
#include <stdlib.h>

int main(){
    srand(time(NULL));

    int r = 5 + rand() % 16;
    int c = 5 + rand() % 16;

    printf("R=%d C=%d\n\n", r, c);

    char bordo = c<10 ? '*' : '#';
    char riempimento = r%2==0 ? bordo : ' ';

    for(int riga = 1; riga <= r; riga++){
        for(int colonna = 1; colonna <= c; colonna++)
            if(riga==1 || colonna==1 || riga==r || colonna==c)
                printf("%c", bordo);
            else
                printf("%c", riempimento);
        printf("\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.4.13 Disegno di figure geometriche

Leggere un numero intero N e visualizzare quattro diverse figure geometriche basate su asterischi e caratteri misti.

Obiettivo

Il programma deve leggere un intero N e stampare le seguenti figure:

1. Un quadrato pieno di asterischi di lato N
2. Lo stesso quadrato con solo i lati (versione vuota)
3. Un triangolo isoscele rettangolo di lato N
4. Un quadrato di lato N con caratteri alternati `*` e `+`

Descrizione

Ogni figura richiede una logica diversa per decidere quale carattere stampare in ogni posizione:

1. **Quadrato pieno**: ogni cella contiene `*`.
2. **Quadrato vuoto**: si stampa `*` solo se la cella è sul bordo (prima/ultima riga o prima/ultima colonna), altrimenti uno spazio.
3. **Triangolo isoscele rettangolo**: alla riga i (partendo da 0) si stampano i+1 asterischi. La figura cresce verso destra scendendo di riga in riga.
4. **Quadrato con caratteri misti**: il carattere varia in base alla posizione. Ad esempio, si usa `*` se la somma degli indici i+j è pari, `+` se è dispari (o viceversa), ottenendo un effetto a scacchiera.

Suggerimenti

- Struttura il programma in quattro sezioni distinte, ognuna con il proprio doppio ciclo `for`.
- Per il triangolo, il ciclo interno va da 0 a i (compreso), dove i è l'indice di riga.
- Per il quadrato con caratteri misti, sfrutta la parità di `(i + j)` per alternare i caratteri.
- Separa le figure con una riga vuota e un'intestazione (es. "Figura 1:") per rendere l'output più chiaro.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int main(void) {
    int lato;

    do{
        printf("\nDi quanto deve essere il lato? ");
        scanf("%d", &lato);
        if( lato<1 )
            printf("ERRORE: Il lato deve valere almeno 1!");
    }while( lato<1 );

    printf("\n");

    for(int r=1; r<=lato; r++){
        for(int c=1; c<=lato; c++)
            printf("*");
        printf("\n");
    }

    printf("\n");

    for(int r=1; r<=lato; r++){
        for(int c=1; c<=lato; c++)
            if(r==1 || r==lato || c==1 || c==lato)
                printf("*");
            else
                printf(" ");
        printf("\n");
    }

    printf("\n");

    for(int r=1; r<=lato; r++){
        for(int c=1; c<=r; c++)
            printf("*");
        printf("\n");
    }

    printf("\n");

    for(int r=1; r<=lato; r++){
        for(int c=1; c<=lato; c++)
            if(r>=c)
                printf("*");
            else
                printf("+");
        printf("\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.4.14 Stampa numero binario

Stampare il valore in binario di un numero positivo chiesto all'utente, visualizzando anche le divisioni successive.

Obiettivo

Il programma deve richiedere un numero intero positivo e mostrare, passo dopo passo, le divisioni successive per 2 utilizzate per la conversione, quindi stampare il risultato in binario.

Descrizione

La conversione di un numero decimale in binario si ottiene dividendo ripetutamente il numero per 2 e raccogliendo i resti. I resti, letti dal basso verso l'alto, formano la rappresentazione binaria.

Ad esempio, per convertire 13:

Divisione	Quoziente	Resto
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Leggendo i resti dal basso: 13 in binario è **1101**.

Poiché i resti vanno letti in ordine inverso rispetto a come vengono calcolati, è necessario memorizzarli (ad esempio in un vettore) prima di stamparli.

Suggerimenti

- Usa un ciclo `do-while` per la validazione: accetta solo valori maggiori di 0.
- Stampa ogni passo della divisione (numero, quoziente, resto) durante il ciclo di conversione.
- Memorizza i resti in un vettore e stampali in ordine inverso al termine del ciclo.
- La dimensione massima del vettore può essere 32 (sufficienti per un intero a 32 bit).

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int main(){
    int numero;

    do {
        printf("Inserire un numero positivo: ");
        scanf("%d", &numero);
        if(numero < 0)
            printf("ERRORE: il numero deve essere positivo.\n\n");
    } while(numero < 0);

    // Stampo le divisioni successive e conto da quante cifre è composto il numero
    int cifre=0;
    for(int n = numero; n>0; n/=2, cifre++)
        printf("%5d | %d\n", n, n%2);
    printf("%5d | \n\n", 0);
    if(numero==0)
        cifre=1;

    printf("%d in binario vale: ", numero);
    for(int i = cifre-1; i>=0; i--) // Stampo ciascuna cifra
        printf("%d", 1<<i & numero && 1);
    printf("\n");

    return 0;
}
```


2025 Daniele Prevedello

4.5 Calcolo numerico

4.5.1 Stampa divisori di un numero

Chiedere un numero intero strettamente positivo e stampare tutti i suoi divisori.

Obiettivo

Il programma deve richiedere un numero intero strettamente positivo (con validazione dell'input) e stampare tutti i valori interi che lo dividono esattamente, cioè con resto pari a zero.

Descrizione

Un divisore di un numero N è un intero d tale che N % d == 0. Per trovare tutti i divisori basta scorrere con un ciclo tutti i valori da 1 fino a N e verificare, per ognuno, se la divisione è esatta.

Si noti che 1 e N stesso sono sempre divisori di N. Se gli unici divisori sono proprio 1 e N, allora N è un numero primo (concetto che verrà approfondito nell'esercizio successivo).

Un'ottimizzazione possibile, non obbligatoria, consiste nel cercare i divisori solo fino a √N, poiché i divisori si presentano sempre a coppie.

Suggerimenti

- Usa un ciclo `do-while` per la validazione: ripeti la richiesta finché l'utente non inserisce un valore maggiore di 0.
- All'interno del ciclo `for` da 1 a N, controlla la condizione `N % d == 0` con un `if` e stampa d.
- Conta i divisori trovati con una variabile contatore: se alla fine vale 2 (solo 1 e N), il numero è primo.
- Per numeri molto grandi, considera di fermare la ricerca a √N e stampare entrambi gli elementi della coppia.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    do {
        printf("Inserire un numero (strettamente positivo): ");
        scanf("%d", &numero);
        if(numero <= 0)
            printf("ERRORE: il numero deve essere strettamente positivo.\n\n");
    } while(numero <= 0);

    printf("\nI divisori di %d sono: ", numero);
    for(int div=1; div<=numero; div++)
        if(numero%div==0)
            printf("%d ", div);
    printf("\n");

    return 0;
}
```


2025 Daniele Prevedello

4.5.2 Verifica numero primo

Inserito un numero intero strettamente positivo, verificare se è primo.

Obiettivo

Il programma deve richiedere un numero intero strettamente positivo (con validazione dell'input) e comunicare se è primo oppure no.

Descrizione

Un numero intero N maggiore di 1 è **primo** se i suoi unici divisori sono 1 e N stesso. Per convenzione, 1 non è considerato un numero primo.

L'algoritmo di verifica più semplice consiste nel cercare divisori nell'intervallo [2, N-1]: se ne viene trovato almeno uno, il numero non è primo. È possibile ottimizzare fermando la ricerca a \(\lfloor\sqrt{N}\rfloor\): se N non ha divisori fino alla sua radice quadrata, non ne avrà nemmeno oltre. Questa ottimizzazione riduce notevolmente il numero di iterazioni per numeri grandi.

I primi numeri primi sono: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ...

Suggerimenti

- Usa un ciclo `do-while` per la validazione: accetta solo valori maggiori di 0.
- Gestisci separatamente il caso N == 1 (non primo per definizione).
- Nel ciclo di verifica, puoi fermarti a `d * d <= N` invece di arrivare a N-1: è molto più efficiente.
- Usa una variabile booleana (o un intero usato come flag) per memorizzare l'esito della verifica e stampare il risultato dopo il ciclo.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    do {
        printf("Inserire un numero (strettamente positivo): ");
        scanf("%d", &numero);
        if(numero <= 0)
            printf("ERRORE: il numero deve essere strettamente positivo.\n\n");
    } while(numero <= 0);

    int divisori = 0;
    for(int div=1; div<=numero; div++)
        if(numero%div==0)
            divisori++;

    if(divisori == 2)
        printf("Il numero è primo!\n");
    else
        printf("Il numero non è primo!\n");

    return 0;
}
```


2025 Daniele Prevedello

4.5.3 Calcolo del fattoriale

Calcolare il fattoriale di un numero chiesto all'utente, controllando l'ammissibilità dell'input.

Obiettivo

Il programma deve richiedere un numero intero non negativo, verificare che sia ammissibile e calcolarne il fattoriale.

Descrizione

Il fattoriale di un numero è indicato con il simbolo **n!** e indica il prodotto di tutti i numeri naturali (escluso lo zero) minori o uguali al numero dato. Ad esempio: \(6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \). L'unica eccezione è **0!** che per definizione vale **1**. Il fattoriale è definito solo per numeri naturali (positivi).

Il calcolo si esegue con un ciclo `for` che moltiplica progressivamente i valori da 1 fino a n. Poiché il fattoriale cresce molto rapidamente (già 13! supera il massimo di un `int` a 32 bit), è opportuno usare il tipo `long long` per memorizzare il risultato.

n	n!
0	1
1	1
5	120
10	3 628 800
12	479 001 600

Suggerimenti

- Usa un ciclo `do-while` per validare l'input: il fattoriale è definito solo per n >= 0.
- Inizializza il risultato a 1 (non a 0), poiché si tratta di una moltiplicazione.
- Usa il tipo `long long` per il risultato e `%lld` in `printf` per stamparlo correttamente.
- Gestisci il caso n == 0 esplicitamente oppure verifica che il ciclo `for` lo gestisca correttamente senza iterazioni (il ciclo `for(i=1; i<=0; i++)` non esegue mai il corpo, lasciando il risultato a 1).

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    do {
        printf("Inserire un numero: ");
        scanf("%d", &numero);
        if(numero < 0)
            printf("ERRORE: per il calcolo del fattoriale il numero deve essere maggiore o uguale a 0.\n\n");
    } while(numero < 0);

    long fattoriale = 1;

    for(int i=1; i<=numero; i++)
        fattoriale*=i;

    printf("\nIl fattoriale di %d vale: %ld\n", numero, fattoriale);

    return 0;
}
```


2025 Daniele Prevedello

4.5.4 Sequenza di Fibonacci

Calcolare l'n-esimo numero di Fibonacci con n chiesto all'utente e validato.

Obiettivo

Il programma deve richiedere un numero intero positivo n (con validazione) e calcolare l'n-esimo termine della successione di Fibonacci.

Descrizione

Leonardo Pisano detto "filio Bonacci" o Fibonacci (Pisa 1175 ca. – 1240 ca.) fu un famoso matematico italiano. La successione di Fibonacci nacque da un problema concreto proposto dall'Imperatore Federico II di Svevia a Pisa nel 1223: **"Quante coppie di conigli si ottengono in un anno, supponendo che ogni coppia dia alla luce un'altra coppia ogni mese e che le coppie più giovani siano in grado di riprodursi già al secondo mese di vita?"**.

La serie è: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ... dove ogni numero si ottiene come somma dei due precedenti.

Il calcolo iterativo richiede solo tre variabili: i due termini precedenti e quello corrente, aggiornati ad ogni passo del ciclo.

Suggerimenti

- Usa un ciclo `do-while` per validare n: accetta solo valori maggiori di 0.
- Gestisci separatamente i casi base: F(1) = 1 e F(2) = 1, prima di entrare nel ciclo.
- Nel ciclo `for` da 3 a n, calcola il termine corrente come somma dei due precedenti e aggiorna le variabili di appoggio.
- Usa il tipo `long long` per gestire valori di n elevati senza overflow.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    int numero;

    do {
        printf("Inserire un numero: ");
        scanf("%d", &numero);
        if(numero <= 0)
            printf("ERRORE: per il calcolo di Fibonacci il numero deve essere strettamente positivo.\n\n");
    } while(numero <= 0);

    int fibonacci = 1, n_1 = 1, n_2 = 0;
    for(int i=2; i<=numero; i++){
        fibonacci = n_1 + n_2;
        n_2 = n_1;
        n_1 = fibonacci;
    }

    printf("Il %d numero di Fibonacci è: %d\n", numero, fibonacci);

    return 0;
}
```


2025 Daniele Prevedello

4.6 Funzioni e ricorsione

4.6.1 Calcolo radice quadrata

Calcolare la radice quadrata di un numero tramite il metodo babilonese, creando una funzione apposita.

Obiettivo

Il programma deve richiedere un numero reale non negativo e calcolarne la radice quadrata utilizzando il metodo babilonese, implementato in una funzione dedicata.

Descrizione

Il **metodo babilonese** è uno dei primi metodi iterativi per il calcolo della radice quadrata. La formula iterativa è:

$$ radQ(\alpha) = \left\{ \begin{align} r_0 &= \frac{\alpha}{2} \\ r_{n+1} &= \frac{1}{2}\left(r_n + \frac{\alpha}{r_n}\right) \end{align} \right. $$

Partendo da una stima iniziale $r_0 = \frac{\alpha}{2}$, ogni iterazione produce un'approssimazione migliore. Il processo si ripete finché la differenza tra due approssimazioni successive è inferiore a una soglia di precisione (ad esempio 10^{-9}).

Il metodo converge molto rapidamente: bastano poche iterazioni per ottenere un risultato preciso anche per numeri grandi.

Suggerimenti

- Crea una funzione `double radice(double alpha)` che implementa il metodo babilonese e restituisce il risultato.
- Come condizione di stop usa `fabs(r_new - r_old) > 1e-9` (dalla libreria `<math.h>`).
- Gestisci il caso $\alpha = 0$ separatamente per evitare divisioni per zero.
- Confronta il risultato con la funzione `sqrt()` di `<math.h>` per verificare la correttezza.

Soluzione

C

```c
#include <stdio.h>

double radQuadrata(double valore);

int main(void) {
    float numero;

    do {
        printf("Inserire il numero del quale si vuole conoscere la radice quadrata: ");
        scanf("%f", &numero);
        if(numero < 0)
            printf("ERRORE: per il calcolo della radice il numero deve essere positivo.\n\n");
    } while(numero < 0);

    printf("\nLa radice vale: %.10f\n", radQuadrata(numero));

    return 0;
}

double radQuadrata(double valore){
    double radice = 1;
    for(int i=0; i<100; i++)
        radice = (radice + valore / radice) / 2;
    return radice;
}
```


2025 Daniele Prevedello

4.6.2 Calcolo MCD

Letti due numeri interi, calcolare il massimo comun divisore (MCD) tramite l'algoritmo di Euclide, creando un'apposita funzione.

Obiettivo

Il programma deve richiedere due numeri interi positivi e calcolarne il massimo comun divisore utilizzando l'algoritmo di Euclide, implementato in una funzione dedicata.

Descrizione

L'algoritmo di Euclide è un metodo per trovare il massimo comune divisore tra due numeri interi: anziché calcolare tutti i divisori, si divide a per b ottenendo il resto r, poi si calcola l'MCD tra b ed r, e si procede finché il resto è 0. L'ultimo divisore non nullo è l'MCD cercato.

Ad esempio, per calcolare MCD(48, 18):

Passo	a	b	r = a % b
1	48	18	12
2	18	12	6
3	12	6	0

Il risultato è **MCD(48, 18) = 6**.

L'algoritmo funziona perché MCD(a, b) = MCD(b, a % b), proprietà dimostrabile con semplici argomenti di divisibilità.

Suggerimenti

- Crea una funzione `int mcd(int a, int b)` che implementa il ciclo `while (b != 0)`.
- All'interno del ciclo, aggiorna le variabili: `int r = a % b; a = b; b = r;`.
- Gestisci il caso in cui uno dei due numeri valga 0: per convenzione MCD(a, 0) = a.
- Questa funzione sarà riutilizzata nell'esercizio sul minimo comune multiplo con Euclide.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int mcd(int n1, int n2);

int main(void) {
    int numero1, numero2;

    do {
        printf("Inserire il primo numero con il qualse si vuole calcolare l'MCD: ");
        scanf("%d", &numero1);
        if(numero1 <= 0)
            printf("ERRORE: per il calcolo dell'MCD il numero deve essere strettamente positivo.\n\n");
    } while(numero1 < 0);

    do {
        printf("Inserire il secondo numero con il qualse si vuole calcolare l'MCD: ");
        scanf("%d", &numero2);
        if(numero2 <= 0)
            printf("ERRORE: per il calcolo dell'MCD il numero deve essere strettamente positivo.\n\n");
    } while(numero2 < 0);

    printf("\nIl massimo comun divisore fra %d e %d è %d\n", numero1, numero2, mcd(numero1, numero2));

    return 0;
}

int mcd(int n1, int n2){
    int a=n1, b=n2, r;

    do {
        r = a % b;
        a = b;
        b = r;
    } while(r != 0);

    return a;
}
```


2025 Daniele Prevedello

4.6.3 Calcolo mcm (base)

Calcolare il minimo comune multiplo (mcm) di due numeri interi positivi tramite apposita funzione.

Obiettivo

Il programma deve richiedere due numeri interi positivi e calcolarne il minimo comune multiplo con un approccio diretto basato sui multipli, implementato in una funzione dedicata.

Descrizione

Dati due numeri interi N1 ed N2, il minimo comune multiplo è il più piccolo numero M divisibile sia per N1 che per N2. Ipotizzando che N1 sia più grande di N2, l'mcm è il primo multiplo di N1 che risulta divisibile per N2 (con resto pari a zero).

Ad esempio, per calcolare mcm(12, 8): si scorrono i multipli di 12 (12, 24, 36, ...) e si verifica quale è il primo divisibile per 8. Il primo è 24, quindi **mcm(12, 8) = 24**.

Questo approccio è semplice da implementare ma può risultare lento per numeri grandi. L'esercizio successivo mostra un metodo più efficiente basato sull'algoritmo di Euclide.

Suggerimenti

- Crea una funzione `int mcm(int n1, int n2)` che scorre i multipli del maggiore tra n1 e n2.
- Prima di entrare nel ciclo, determina quale dei due numeri è il maggiore.
- Il ciclo parte dal valore maggiore e avanza di quel valore ad ogni passo: `m += maggiore`.
- Imposta una condizione di uscita sicura (ad esempio un numero massimo di iterazioni) per evitare cicli infiniti in caso di input anomali.

Soluzione

C

```c
#include <stdio.h>

int mcm(int n1, int n2);

int main(void) {
    int numero1, numero2;

    do {
        printf("Inserire il primo numero con il qualse si vuole calcolare l'mcm: ");
        scanf("%d", &numero1);
        if(numero1 <= 0)
            printf("ERRORE: per il calcolo dell'mcm il numero deve essere strettamente positivo.\n\n");
    } while(numero1 <= 0);

    do {
        printf("Inserire il secondo numero con il qualse si vuole calcolare l'mcm: ");
        scanf("%d", &numero2);
        if(numero2 <= 0)
            printf("ERRORE: per il calcolo dell'mcm il numero deve essere strettamente positivo.\n\n");
    } while(numero2 <= 0);

    printf("\nIl minimo comune multiplo fra %d e %d è %d\n", numero1, numero2, mcm(numero1, numero2));

    return 0;
}

int mcm(int n1, int n2){
    int a=n1, b=n2;
    if(n2>n1){ // In "a" c'è sempre il più grande tra n1 ed n2
        a=n2;
        b=n1;
    }

    for(int i=1; i<=a; i++)
        if(b*i % a == 0)
            return b*i;

```


2025 Daniele Prevedello

```
        return 0; //TMCH
    }
```


2025 Daniele Prevedello

4.6.4 Calcolo mcm con Euclide

Calcolare il minimo comune multiplo (mcm) di due numeri interi positivi sfruttando l'algoritmo di Euclide.

Obiettivo

Il programma deve richiedere due numeri interi positivi e calcolarne il minimo comune multiplo utilizzando la relazione con il MCD, implementando le apposite funzioni.

Descrizione

Dati due numeri interi N1 ed N2, l'mcm si può ottenere dividendo il loro prodotto per il MCD:

\[mcm(N1, N2) = \frac{N1 \cdot N2}{MCD(N1, N2)} \]

Questo approccio è molto più efficiente di quello basato sui multipli, perché sfrutta l'algoritmo di Euclide che converge in pochi passi. Ad esempio:

\[mcm(12, 8) = \frac{12 \cdot 8}{MCD(12, 8)} = \frac{96}{4} = 24\]

È buona pratica eseguire prima la divisione e poi la moltiplicazione (`n1 / mcd(n1, n2) * n2`) per ridurre il rischio di overflow con numeri grandi.

Suggerimenti

- Riutilizza (o riscrivi) la funzione `mcd(int a, int b)` dell'esercizio precedente.
- Crea una funzione `int mcm(int n1, int n2)` che chiama `mcd` e applica la formula.
- Per ridurre il rischio di overflow calcola `(n1 / mcd(n1, n2)) * n2` invece di `(n1 * n2) / mcd(n1, n2)`.
- Verifica il risultato confrontandolo con quello dell'esercizio 44 su alcuni esempi.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int mcm(int n1, int n2);
int mcd(int n1, int n2);

int main(void) {
    int numero1, numero2;

    do {
        printf("Inserire il primo numero con il qualse si vuole calcolare l'mcm: ");
        scanf("%d", &numero1);
        if(numero1 <= 0)
            printf("ERRORE: per il calcolo dell'mcm il numero deve essere strettamente positivo.\n\n");
    } while(numero1 <= 0);

    do {
        printf("Inserire il secondo numero con il qualse si vuole calcolare l'mcm: ");
        scanf("%d", &numero2);
        if(numero2 <= 0)
            printf("ERRORE: per il calcolo dell'mcm il numero deve essere strettamente positivo.\n\n");
    } while(numero2 <= 0);

    printf("\nIl minimo comune multiplo fra %d e %d è %d\n", numero1, numero2, mcm(numero1, numero2));

    return 0;
}

int mcm(int n1, int n2){
    return n1*n2 / mcd(n1, n2);
}

int mcd(int n1, int n2){
    int a=n1, b=n2, r;

    do {
        r = a % b;
        a = b;
        b = r;
    } while(r != 0);

    return a;
}
```


2025 Daniele Prevedello

4.6.5 Calcolo del numero di Nepero

Calcolare il numero di Nepero e tramite la sua definizione come serie infinita.

Obiettivo

Il programma deve calcolare un'approssimazione del numero **e** sommando i primi 21 termini (da n=0 a n=20) della sua serie di definizione, usando il tipo `long double` per massimizzare la precisione.

Descrizione

Il numero **e** può essere definito come la serie:

\[e := \sum_{n=0}^\infty \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \ldots \]

A causa dei limiti di rappresentazione dei tipi `float` e `double`, si usa il tipo `long double` (16 byte). Il calcolo si ferma al 20° termine poiché oltre non si ottiene beneficio per i limiti di precisione.

Il valore atteso è: **e ≈ 2.71828182845904523536...**

Ogni termine della serie è \(\frac{1}{n!}\): invece di ricalcolare il fattoriale da zero ad ogni passo, è possibile aggiornare il termine corrente moltiplicandolo per \(\frac{1}{n}\) ad ogni iterazione, rendendo il calcolo molto più efficiente.

Suggerimenti

- Usa il tipo `long double` per le variabili e `%Lf` (con L maiuscola) in `printf` per stamparle.
- Ottimizza il calcolo aggiornando il termine corrente: `termine /= n` ad ogni iterazione, evitando di ricalcolare n! da zero.
- Stampa il valore di e con almeno 15 cifre decimali per apprezzare la precisione ottenuta.
- Confronta il risultato con la costante `M_E` della libreria `<math.h>`.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    long double e = 1;
    long double fatt = 1;

    for( int i=1; i<=20; i++ ){
        fatt *= (1.L/i);
        e += fatt;
    }

    printf("Il valore di e è pari a: %.20Lf\n", e);

    return 0;
}
```


2025 Daniele Prevedello

4.6.6 Calcolo del fattoriale (ricorsivo)

Calcolare il fattoriale di un numero chiesto all'utente utilizzando la ricorsione, con controllo dell'ammissibilità dell'input.

Obiettivo

Il programma deve richiedere un numero intero non negativo, verificarne l'ammissibilità e calcolarne il fattoriale tramite una funzione ricorsiva.

Descrizione

Il fattoriale è indicato con **n!** e vale il prodotto di tutti i naturali da 1 a n (con 0! = 1 per definizione).

Una funzione ricorsiva richiama sé stessa (**ricorsione diretta**) oppure richiama una funzione che a sua volta la richiama (**ricorsione indiretta**). Affinché il procedimento abbia fine devono esistere dei **casi base** per cui la funzione restituisce un valore senza richiamare sé stessa, e ogni chiamata ricorsiva deve avvicinare i parametri ai casi base.

Nel caso del fattoriale:

- **Caso base**: `fattoriale(0) = 1`
- **Caso ricorsivo**: `fattoriale(n) = n * fattoriale(n - 1)`

Ad ogni chiamata ricorsiva n diminuisce di 1, avvicinandosi al caso base. Lo stack delle chiamate cresce fino a raggiungere il caso base, dopodiché i valori vengono restituiti risalendo la catena.

Suggerimenti

- Definisci una funzione `long long fattoriale(int n)` che implementa i due casi con un `if-else`.
- Il caso base è `if (n == 0) return 1`, il caso ricorsivo è `return n * fattoriale(n - 1)`.
- Usa il tipo `long long` per il valore di ritorno, per gestire risultati grandi senza overflow.
- Usa un ciclo `do-while` per la validazione dell'input: accetta solo valori n >= 0.

Soluzione

C

```c
#include <stdio.h>

long fattoriale(int n);

int main(void) {
    int numero;

    do {
        printf("Inserire un numero: ");
        scanf("%d", &numero);
        if(numero < 0)
            printf("ERRORE: per il calcolo del fattoriale il numero deve essere maggiore o uguale a 0.\n\n");
    } while(numero < 0);

    printf("\nIl fattoriale di %d vale: %ld\n", numero, fattoriale(numero));

    return 0;
}

long fattoriale(int n){
    if(n < 0)
        return -1;
    if(n == 0)
        return 1;
    return n * fattoriale(n-1);
}
```


2025 Daniele Prevedello

4.6.7 Sequenza di Fibonacci (ricorsivo)

Calcolare l'n-esimo numero di Fibonacci con n chiesto all'utente e validato, utilizzando le funzioni ricorsive.

Obiettivo

Il programma deve richiedere un numero intero positivo n (con validazione) e calcolare l'n-esimo termine della successione di Fibonacci tramite una funzione ricorsiva.

Descrizione

La successione di Fibonacci (1, 1, 2, 3, 5, 8, ...) nasce da un problema proposto da Federico II di Svevia nel 1223 e fu risolta da Leonardo Pisano detto Fibonacci.

Una funzione ricorsiva richiama sé stessa (**ricorsione diretta**) o tramite un'altra funzione (**ricorsione indiretta**). Sono necessari **casi base** e ogni chiamata deve avvicinare i parametri ai casi base.

Nel caso di Fibonacci:

- **Casi base**: `F(1) = 1` e `F(2) = 1`
- **Caso ricorsivo**: `F(n) = F(n-1) + F(n-2)`

A differenza della versione iterativa, quella ricorsiva è più elegante ma meno efficiente: per calcolare F(n) vengono effettuate un numero esponenziale di chiamate, poiché molti sottoproblemi vengono ricalcolati più volte.

Suggerimenti

- Definisci una funzione `long long fibonacci(int n)` con due casi base (`if (n <= 2) return 1`) e il caso ricorsivo.
- Usa un ciclo `do-while` per la validazione: accetta solo valori n >= 1.
- Testa il programma con valori di n non troppo grandi (fino a circa 40): oltre, il tempo di calcolo cresce rapidamente.
- Confronta i tempi di esecuzione con la versione iterativa per apprezzare la differenza di efficienza.

Soluzione

C

```c
#include <stdio.h>

int fibonacci(int n);

int main(void) {
    int numero;

    do {
        printf("Inserire un numero: ");
        scanf("%d", &numero);
        if(numero <= 0)
            printf("ERRORE: per il calcolo di Fibonacci il numero deve essere strettamente positivo.\n\n");
    } while(numero <= 0);

    printf("Il %d° numero di Fibonacci è: %d\n", numero, fibonacci(numero));

    return 0;
}

int fibonacci(int n){
    if(n < 0)
        return -1;
    if(n <= 1)
        return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```


2025 Daniele Prevedello

4.7 Enum

4.7.1 Analisi sequenza di numeri

Leggere N numeri interi da tastiera e calcolarne statistiche su segno, parità e andamento.

Obiettivo

Il programma deve chiedere all'utente quanti numeri inserire (N), leggere la sequenza e stampare:

1. Quanti numeri sono **positivi**, **nulli** e **negativi**
2. Quanti sono **pari** e **dispari**
3. Se la sequenza è **crescente**, **decrescente** oppure **né crescente né decrescente**

Descrizione

Dati N numeri interi letti da tastiera (con N scelto dall'utente) si vogliono calcolare e stampare: 1) quanti sono i numeri **positivi**, **nulli** e **negativi**; 2) quanti sono i numeri **pari** e **dispari**; 3) se la sequenza è **crescente**, **decrescente** oppure **né crescente né decrescente**.

Per analizzare l'andamento della sequenza è utile confrontare ogni elemento con il precedente. Si possono usare un tipo enumerativo (`enum`) o variabili flag per tenere traccia dell'andamento rilevato finora. Una sequenza è crescente se ogni elemento è strettamente maggiore del precedente, decrescente se strettamente minore; altrimenti non è né l'una né l'altra.

Suggerimenti

- Usa un `enum` (ad esempio `CRESCENTE`, `DECRESCENTE`, `MISTO`) per rappresentare lo stato dell'andamento in modo leggibile.
- Leggi il primo numero separatamente e inizializza le variabili di confronto; poi usa un ciclo `for` per i restanti N-1 numeri.
- Aggiorna i contatori di segno e parità ad ogni iterazione con `if-else` e l'operatore `%`.
- Gestisci la validazione di N con un `do-while`: accetta solo valori N >= 1.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

typedef enum tipologia {
    crescente,
    decrescente,
    misto,
    indefinito
} tipologia;

int main(void) {
    int n;

    do {
        printf("Quanti numeri vuoi inserire? ");
        scanf("%d", &n);
        if(n < 1)
            printf("ERRORE: devi inserire almeno un numero.\n\n");
    } while(n < 1);

    int positivi=0, nulli=0, negativi=0;
    int pari=0, dispari=0;
    int prec;
    tipologia tipo = indefinito;

    for(int i=0; i<n; i++){
        int numero;
        printf("%d) inserire un numero: ", i+1);
        scanf("%d", &numero);

        // Positivi, nulli e negativi
        if(numero < 0)
            negativi++;
        else if(numero == 0)
            nulli++;
        else
            positivi++;

        // Pari o dispari
        if(numero%2 == 0)
            pari++;
        else
            dispari++;

        // Sequenza crescente, decrescente o né crescente né decrescente
        if(i>0){
            if(tipo == indefinito){
                if(prec < numero)
                    tipo=crescente;
                if(prec > numero)
                    tipo=decrescente;
            } else if(tipo != misto){
                if(prec < numero && tipo == decrescente)
                    tipo=misto;
                if(prec > numero && tipo == crescente)
                    tipo=misto;
            }
        }
        prec=numero;
    }

    printf("\nValori positivi: %d\nValori nulli: %d\nValori negativi: %d\n\n", positivi, nulli, negativi);
    printf("Numeri pari: %d\nNumeri dispari: %d\n\n", pari, dispari);

    switch(tipo){
        case indefinito:
            printf("Nulla può essere detto sulla sequenza.\n");
            break;
        case crescente:
            printf("La sequenza è crescente.\n");
            break;
        case decrescente:
            printf("La sequenza è decrescente.\n");
            break;
        case misto:
            printf("La sequenza è né crescente né decrescente.\n");
    }

    return 0;
}
```


2025 Daniele Prevedello

4.8 Stringhe e vettori

4.8.1 Conversione in maiuscolo

Chiedere all'utente una parola e convertirla interamente in maiuscolo.

Obiettivo

Il programma deve leggere una parola inserita dall'utente e stamparla con tutte le lettere convertite in maiuscolo.

Descrizione

In C le stringhe sono vettori di caratteri terminati dal carattere nullo `'\0'`. Per scorrere una stringa si usa un ciclo che avanza fino a incontrare il terminatore.

La conversione da minuscolo a maiuscolo sfrutta la codifica ASCII: le lettere minuscole (`'a'` – `'z'`) e le lettere maiuscole (`'A'` – `'Z'`) distano esattamente 32 posizioni. Una lettera minuscola si riconosce con la condizione `c >= 'a' && c <= 'z'` e si converte sottraendo 32 (oppure usando la funzione `toupper()` della libreria `<ctype.h>`).

I caratteri che non sono lettere minuscole (cifre, spazi, maiuscole già presenti) vengono lasciati invariati.

Suggerimenti

- Leggi la parola con `scanf("%s", parola)`, che si ferma al primo spazio.
- Usa un ciclo `for` con indice intero oppure un ciclo `while (parola[i] != '\0')` per scorrere i caratteri.
- Applica la conversione con `toupper()` (includi `<ctype.h>`) oppure con la condizione `if (c >= 'a' && c <= 'z') c -= 32`.
- Stampa la parola convertita carattere per carattere oppure direttamente con `printf("%s", parola)` dopo la modifica in-place.

Soluzione

C

```c
#include <stdio.h>

int main(void) {
    char parola[100];

    printf("Inserire una parola: ");
    scanf("%s", parola);

    int scostamento = 'A' - 'a';
    for(int i=0; i<100 && parola[i]!='\0'; i++)
        if(parola[i]>='a' && parola[i]<='z')
            parola[i] = parola[i] + scostamento;

    printf("Parola in maiuscolo: %s\n", parola);

    return 0;
}
```


2025 Daniele Prevedello

4.8.2 Parola palindroma

Chiedere all'utente una parola e verificare che sia palindroma.

Obiettivo

Il programma deve leggere una parola inserita dall'utente e comunicare se è palindroma oppure no.

Descrizione

Una parola è **palindroma** se si legge allo stesso modo sia da sinistra verso destra che da destra verso sinistra. Esempi classici: *radar*, *kayak*, *anna*, *level*.

Per verificarlo si confrontano i caratteri simmetrici rispetto al centro della parola: il primo con l'ultimo, il secondo con il penultimo, e così via. Il confronto si ferma al carattere centrale (o alla coppia centrale se la lunghezza è pari). Se tutti i confronti hanno esito positivo, la parola è palindroma; basta un solo disaccordo per escluderlo.

In C la lunghezza di una stringa si ottiene con la funzione `strlen()` della libreria `<string.h>`.

Suggerimenti

- Leggi la parola con `scanf("%s", parola)` e calcolane la lunghezza con `strlen()`.
- Usa un ciclo `for` che va da 0 a `lunghezza/2 - 1` e confronta `parola[i]` con `parola[lunghezza - 1 - i]`.
- Usa una variabile flag (es. `int palindroma = 1`) che imposti a 0 al primo carattere diverso, uscendo dal ciclo con `break`.
- Stampa l'esito dopo il ciclo controllando il valore del flag.

Soluzione

C

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char parola[100];

    printf("Inserire una parola: ");
    scanf("%s", parola);

    int len = strlen(parola), palindroma = 1;
    for(int i=0; i<len/2 && palindroma; i++)
        if(parola[i] != parola[len-i-1])
            palindroma = 0;

    printf("La parola %s palindroma\n", palindroma ? "è" : "NON è");

    return 0;
}
```


2025 Daniele Prevedello

4.8.3 Algoritmo di Cesare

Codificare una frase secondo l'algoritmo di Cesare.

Obiettivo

Il programma deve leggere una frase inserita dall'utente e una chiave di cifratura (un numero intero), quindi codificare la frase applicando la cifratura di Cesare.

Descrizione

La **cifratura di Cesare** è uno dei più antichi algoritmi crittografici, usata da Giulio Cesare per le comunicazioni militari. Ogni lettera del messaggio viene sostituita con la lettera che si trova k posizioni più avanti nell'alfabeto, dove k è la chiave di cifratura. I caratteri che non sono lettere (spazi, cifre, punteggiatura) vengono lasciati invariati.

Ad esempio, con chiave k = 3:

Originale	**A**	**B**	**C**	**...**	**X**	**Y**
Cifrato	D	E	F	...	A	B

La lettera **A** diventa **D**, la **Z** diventa **C** (si "avvolge" tornando all'inizio dell'alfabeto). La stessa logica si applica separatamente alle lettere minuscole e a quelle maiuscole.

La decifratura si ottiene usando una chiave negativa oppure sottraendo k invece di sommare.

Suggerimenti

- Leggi la frase con `fgets(frase, sizeof(frase), stdin)` per gestire anche gli spazi.
- Per ogni carattere, verifica se è una lettera maiuscola (`>= 'A' && <= 'Z'`) o minuscola (`>= 'a' && <= 'z'`) e applica lo spostamento solo in quel caso.
- Per gestire lo "avvolgimento" usa l'operatore modulo: per le maiuscole `(c - 'A' + k) % 26 + 'A'`, in modo analogo per le minuscole.
- Gestisci chiavi negative normalizzando prima: `k = ((k % 26) + 26) % 26`.

Soluzione

C

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char frase[500];

    printf("Inserire una frase da cifrare: ");
    fgets(frase, sizeof(frase), stdin);
    frase[strlen(frase)-1] = '\0';

    for(int i=0; frase[i]!='\0'; i++){
        if( frase[i]>='a' && frase[i]<='z' )
            frase[i] = (frase[i] - 'a' + 3) % 26 + 'a';
        if( frase[i]>='A' && frase[i]<='Z' )
            frase[i] = (frase[i] - 'A' + 3) % 26 + 'A';
    }

    printf("La frase cifrata è: %s\n", frase);

    return 0;
}
```


2025 Daniele Prevedello

4.8.4 Statistiche su una frase

Letta una frase, dire da quanti caratteri, lettere maiuscole/minuscole, vocali/consonanti, cifre, parole è composta.

Obiettivo

Il programma deve leggere una frase inserita dall'utente e stampare le seguenti statistiche:

- numero totale di caratteri (escluso il terminatore `\0`)
- numero di lettere maiuscole e di lettere minuscole
- numero di vocali e di consonanti
- numero di cifre numeriche
- numero di parole

Descrizione

L'analisi di una stringa richiede di scorrere ogni carattere e classificarlo. La libreria `<ctype.h>` mette a disposizione funzioni utili: `isupper()`, `islower()`, `isdigit()`, `isalpha()`, `isspace()` che restituiscono un valore non nullo se la condizione è vera.

Le vocali sono le lettere a, e, i, o, u (maiuscole e minuscole). Le consonanti sono tutte le lettere che non sono vocali.

Il conteggio delle parole si basa sul numero di transizioni da "spazio" a "non spazio": ogni volta che si incontra un carattere non bianco preceduto da uno spazio (o dall'inizio della stringa), si conta una nuova parola.

Suggerimenti

- Leggi la frase con `fgets(frase, sizeof(frase), stdin)` per includere gli spazi.
- Usa un ciclo `for` o `while` con le funzioni di `<ctype.h>` per classificare ogni carattere.
- Per riconoscere le vocali, crea una stringa di riferimento `"aeiouAEIOU"` e usa `strchr()` oppure un `if` con condizioni multiple.
- Per contare le parole, tieni traccia di una variabile booleana `in_parola`: passa a 1 quando trovi un carattere non bianco dopo uno spazio, torna a 0 allo spazio.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char frase[500];

    printf("Inserire una frase: ");
    fgets(frase, sizeof(frase), stdin);
    frase[strlen(frase)-1] = '\0';

    int lunghezza=0, minuscole=0, maiuscole=0;
    int vocali=0, consonanti=0, cifre=0;
    for(int i=0; frase[i]!='\0'; i++){
        lunghezza++;

        if(frase[i]>='a' && frase[i]<='z')
            minuscole++;
        if(frase[i]>='A' && frase[i]<='Z')
            maiuscole++;

        if( (frase[i]>='a' && frase[i]<='z') || (frase[i]>='A' && frase[i]<='Z') ){
            if(frase[i]=='a' || frase[i]=='e' || frase[i]=='i' || frase[i]=='o' || frase[i]=='u')
                vocali++;
            else if(frase[i]=='A' || frase[i]=='E' || frase[i]=='I' || frase[i]=='O' || frase[i]=='U')
                vocali++;
            else
                consonanti++;
        }

        if(frase[i]>='0' && frase[i]<='9')
            cifre++;
    }

    int parole=0, inizio=0;
    for(int i=0; frase[i]!='\0'; i++){
        if( inizio==0 && frase[i]!=' ' && frase[i]!='\t' )
            inizio=1;
        else if( inizio==1 && (frase[i]==' ' || frase[i]=='\t') ){
            inizio=0;
            parole++;
        }
    }
    if(inizio==1)
        parole++;

    printf("La frase è lunga %d caratteri\n", lunghezza);
    printf("Ha %d minuscole e %d maiuscole\n", minuscole, maiuscole);
    printf("Ha %d vocali e %d consonanti\n", vocali, consonanti);
    printf("Ha %d cifre e %d parole\n", cifre, parole);

    return 0;
}
```


2025 Daniele Prevedello

4.8.5 Conversione numero in binario

Convertire un numero positivo chiesto all'utente in binario, utilizzando un vettore come ausilio per il calcolo.

Obiettivo

Il programma deve richiedere un numero intero positivo e stamparne la rappresentazione binaria, usando un vettore per memorizzare le cifre binarie durante il calcolo.

Descrizione

La conversione da decimale a binario si ottiene dividendo ripetutamente il numero per 2 e raccogliendo i resti. I resti, letti in ordine inverso rispetto a come vengono prodotti, formano la rappresentazione binaria.

A differenza dell'esercizio 41, in cui i passaggi intermedi venivano mostrati uno a uno, qui l'obiettivo è raccogliere tutti i resti in un vettore e poi stamparli al contrario, ottenendo direttamente la stringa binaria finale.

Ad esempio, per convertire 13:

Divisione	Quoziente	Resto
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

I resti nel vettore sono [1, 0, 1, 1]; stampati al contrario: **1101**.

Suggerimenti

- Dichiara un vettore di interi di dimensione 32 (sufficiente per un `int` a 32 bit) e un indice inizializzato a 0.
- In un ciclo `while (n > 0)`, salva `n % 2` nel vettore, incrementa l'indice e aggiorna `n = n / 2`.
- Stampa il vettore scorrendo dall'ultimo elemento inserito (indice - 1) fino a 0.
- Usa un ciclo `do-while` per la validazione: accetta solo valori maggiori di 0.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>

int main(){
    int numero;

    do {
        printf("Inserire un numero positivo: ");
        scanf("%d", &numero);
        if(numero < 0)
            printf("ERRORE: il numero deve essere positivo.\n\n");
    } while(numero < 0);

    // Calcolo il valore in binario e conto da quante cifre è composto il numero
    int binario[64] = {0, }, cifre=0;
    for(int n = numero; n>0; n/=2, cifre++)
        binario[cifre] = n%2;
    if(numero==0)
        cifre=1;

    printf("%d in binario vale: ", numero);
    for(int i = cifre-1; i>=0; i--) // Stampo ciascuna cifra
        printf("%d", binario[i]);
    printf("\n");

    return 0;
}
```


2025 Daniele Prevedello

4.8.6 Calcoli su vettori random

Caricare un vettore di n numeri casuali (tra -100 e 100), con n chiesto all'utente (massimo 200), e offrire un menu per stamparli e calcolarne le statistiche.

Obiettivo

Il programma deve chiedere all'utente quanti numeri generare (n, massimo 200), riempire un vettore con numeri casuali nell'intervallo [-100, 100] e presentare un menu che permetta di:

1. Stampare il vettore
2. Calcolare il minimo
3. Calcolare il massimo
4. Calcolare la media
5. Calcolare la moda
6. Calcolare la varianza
7. Uscire

Descrizione

Questo esercizio unisce la generazione di dati casuali, la gestione di un menu con `switch-case` e il calcolo di diverse statistiche descrittive, ciascuna implementata in una funzione dedicata.

- **Minimo e massimo**: si trovano scorrendo il vettore con un ciclo e aggiornando le variabili di appoggio.
- **Media**: somma di tutti gli elementi divisa per n.
- **Moda**: il valore che compare più frequentemente. Si può calcolare contando le occorrenze di ciascun valore in un vettore di frequenze (dimensione 201 per coprire l'intervallo [-100, 100]).
- **Varianza**: media dei quadrati degli scarti dalla media, calcolata come \(\frac{1}{n}\sum_{i=0}^{n-1}(x_i - \bar{x})^2\).

Suggerimenti

- Per generare numeri in [-100, 100] usa la formula `rand() % 201 - 100`.
- Implementa ogni operazione come funzione separata che riceve il vettore e n come parametri.
- Per la moda, usa un vettore di frequenze `freq[201]` dove l'indice è il valore + 100 (per gestire i negativi).
- Il menu può essere gestito con un ciclo `do-while` che si ripete finché l'utente non sceglie di uscire.

Soluzione

C

2025 Daniele Prevedello

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

void stampa(int valori[], int len){
    for(int i=0; i<len; i++)
        printf("%d ", valori[i]);
}

int minimo(int valori[], int len){
    int min = valori[0];
    for(int i=1; i<len; i++)
        if(valori[i] < min)
            min = valori[i];
    return min;
}

int massimo(int valori[], int len){
    int max = valori[0];
    for(int i=1; i<len; i++)
        if(valori[i] > max)
            max = valori[i];
    return max;
}

float media(int valori[], int len){
    float somma = 0;
    for(int i=0; i<len; i++)
        somma += valori[i];
    return somma / len;
}

int moda(int valori[], int len){
    int index, frequenza = 0;
    for(int i=0; i<len; i++){
        int freq = 1;
        for(int j=i+1; j<len; j++)
            if( valori[i] == valori[j] )
                freq++;
        if(freq > frequenza){
            frequenza = freq;
            index = i;
        }
    }
    if( frequenza == 1 )
        return -1;
    return index;
}

int mediana(int valori[], int len){
    return valori[len/2];
}

float varianza(int valori[], int len){
    float avg = media(valori, len);
    float somma;
    for(int i=0; i<len; i++)
        somma += pow(valori[i] - avg, 2);
    return somma / len;
}

int main(void) {
    int numeri[200];
    int n;

    do {
        printf("Quanti numeri vanno generati (da 1 a 200): ");
        scanf("%d", &n);
        if(n < 1 || n > 200)
            printf("ERRORE: il numero deve essere tra 1 e 200.\n\n");
    } while(n < 1 || n > 200);

    srand(time(NULL));
    for(int i=0; i<n; i++)
        numeri[i] = rand() % 201 - 100;

    printf("\nMenu\n");
    printf("1) Stampa\n");
    printf("2) Minimo\n");
    printf("3) Massimo\n");
    printf("4) Media\n");
    printf("5) Moda\n");
    printf("6) Varianza\n");
    printf("7) Termina\n");

    int scelta;
    do{
        printf("Operazione da effettuare? ");
        scanf("%d", &scelta);
        printf("\n");

        int val;
        switch(scelta){
```


2025 Daniele Prevedello